# Smart Medical Insurance Cost Predictor with NGO & Welfare Aid Guidance

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## BACHELOR OF TECHNOLOGY

**In**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**By**


**S. Trividha – 2420030010**

**K. Deepthi – 2420030484**

**M. Swetha – 2420030116**


Under the Esteemed Guidance of

**Ms. B. Prathusha**

**Assistant Professor**

**Department of Computer Science and Engineering**

**AY- 2026-27**

**Semester: Odd**


**Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University**
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

# Declaration

The Project Report entitled "**Smart Medical Insurance Cost Predictor with NGO & Welfare Aid Guidance**" is a record of Bonafide work of **S. Trividha – 2420030010, K. Deepthi – 2420030484, M. Swetha – 2420030116** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

**S. Trividha – 2420030010**

**K. Deepthi – 2420030484**

**M. Swetha – 2420030116**

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE**

**AND ENGINEERING**




# CERTIFICATE

This is to certify that the project report entitled "**Smart Medical Insurance Cost Predictor with NGO & Welfare Aid Guidance**" is a bonafide work done and submitted by **S. Trividha – 2420030010, K. Deepthi – 2420030484, M. Swetha – 2420030116** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), Bowrampet Campus, Hyderabad, during the academic year **2026-2027.**

**Signature of the Guide**

**Signature of the Course Coordinator** **Signature of the HOD**

# ACKNOWLEDGEMENT

The success in this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who assisted us, in one way or another, in completing this project.

Our greatest appreciation goes to our Course Coordinator, and to our guide **Ms. B. Prathusha**, Assistant Professor, Department of Computer Science and Engineering, for her tremendous support, encouragement and guidance throughout this project.

We express our gratitude to the Head of the Department of Computer Science and Engineering for providing us with adequate facilities, ways and means by which we were able to complete this project.

We thank all the members of the teaching and non-teaching staff, and all who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

**S. Trividha – 2420030010**

**K. Deepthi – 2420030484**

**M. Swetha – 2420030116**

# Abstract

Medical treatment in India can cost far more than a family expects, and most cost tools stop at a number. They do not tell a patient who cannot pay which NGO or government scheme may help, or by how much. This project builds one system that does both. It predicts the yearly medical cost for an Indian profile in rupees and then matches the same profile to eligible NGO and welfare schemes, showing the likely aid for each.

Because no public Indian dataset with the required fields exists, the team prepared an India-adapted medical cost dataset of 2,500 records and nine features, with the amounts in ₹. The data was cleaned (75 missing BMI values filled with the median, duplicates checked, outliers examined with the IQR rule), encoded, scaled and explored through histograms, box plots, scatter plots and a correlation heat map. Seven of the nine features were kept after a feature-importance check. Linear Regression, Random Forest and Gradient Boosting were compared on a 20% hold-out set and with 5-fold cross-validation. Gradient Boosting was best with RZ = 0.921 and a mean absolute error of ₹4,683, against 0.899 for Random Forest and 0.871 for Linear Regression. Smoking status was the strongest cost driver (39% of the model's importance), followed by pre-existing condition, city tier, BMI and age.

The aid matcher is rule-based on purpose so that every result can be explained. It checks income, condition type and region against a curated directory of twelve sample NGO and welfare entries and ranks the eligible ones by the likely aid in ₹. Everything is delivered through a Python Flask web dashboard with three tabs: Predict and Find Aid, Analytics and Schemes Directory. Twenty-five automated test cases (TC-01 to TC-25) covering the matching rules, data cleaning, the model and the web API all passed. The work shows how data preprocessing, exploratory analysis and model comparison can be joined to a simple rule engine to give a patient a practical next step.

# Index


| S. No. | Chapters | Topics | Page.no |
|---|---|---|---|
| | | Declaration, Certificate, Acknowledgement | |
| | | Abstract | |
| 1 | **Introduction** | 1.1 Background of the project<br>1.2 Problem statement<br>1.3 Scope of the project<br>1.4 Objectives<br>1.5 Importance of the application<br>1.6 Target users<br>1.7 Literature survey and research gap | 1-3 |
| 2 | **System Requirements** | 2.1 Hardware requirements<br>2.2 Software requirements<br>2.3 Development tools and environment | 4-5 |
| 3 | **Technology Stack** | 3.1 Front-end (HTML, CSS, JavaScript)<br>3.2 Back-end (Python, Flask, scikit-learn)<br>3.3 Data storage (CSV, JSON, joblib)<br>3.4 Version control (Git, GitHub) | 6-7 |
| 4 | **System Architecture** | 4.1 High-level architecture diagram and methodology<br>4.2 Description of components<br>4.3 Deployment architecture | 8-10 |
| 5 | **Design** | 5.1 Wireframes or mockups<br>5.2 Data schema (ER-style diagram)<br>5.3 Data flow diagrams | 11-15 |
| 6 | **Implementation** | 6.1 Module-wise implementation<br>6.2 Front-end logic (UI rendering, state handling)<br>6.3 Back-end logic (routes and services)<br>6.4 Data loading and model persistence<br>6.5 API integration | 16-25 |
| 7 | **Features** | 7.1 List of main features<br>7.2 How each feature works (user and technical view) | 26-29 |
| 8 | **Testing** | 8.1 Functional and API testing<br>8.2 Integration testing<br>8.3 Tools used for testing<br>8.4 Test cases and results (model evaluation, analysis, test table) | 30-33 |
| 9 | **Deployment** | 9.1 Steps to run and deploy<br>9.2 Environment configuration<br>9.3 Hosting | 34 |
| 10 | **Challenges & Limitations** | 10.1 Issues faced during development<br>10.2 Solutions applied<br>10.3 Current limitations | 35-36 |
| 11 | **Future Enhancements** | 11.1 Planned features<br>11.2 Possible integrations and optimizations | 37 |
| 12 | **Conclusion** | 12.1 Summary of the project<br>12.2 What was achieved<br>12.3 Skills learned | 38-39 |
| 13 | **References** | Books, papers, documentation sites used | 40 |
| 14 | **Appendices** | A. Screenshots of the application<br>B. Sample code snippets<br>C. Installation and setup instructions<br>D. User manual | 41-43 |

# CHAPTER 1
# Introduction

## 1.1 Background of the Project

Healthcare in India is becoming more expensive every year. A hospital stay, a surgery or the long treatment of a chronic illness can use up a family's savings within a few weeks. Most families do not know in advance what a year of medical care may cost them, and they find out only when a bill arrives. Studies on medical insurance cost prediction show that a small set of facts about a person, such as age, body mass index (BMI) and whether the person smokes, can explain most of the difference in medical cost [1], [2], [5], [6].

Knowing the likely cost is only half of the problem. India also has many government schemes, charitable trusts and NGOs that give money for treatment, but each one has its own rules about income, illness and place of residence. A patient who cannot pay seldom knows which of them to approach. Cost-prediction tools do not help here, and scheme-recommendation tools [4], [7] do not know what the treatment is likely to cost.

This project, **Smart Medical Insurance Cost Predictor with NGO & Welfare Aid Guidance**, joins the two halves. A machine-learning model estimates the yearly medical cost of a person in rupees (₹). A transparent rule-based engine then compares the person's income, condition and region with a directory of NGO and welfare schemes and lists the schemes the person is eligible for, together with the likely amount of aid. Both results are shown in one web dashboard.

The project is a data science project. For that reason, much of the work and of this report is about the data itself: how it was prepared, cleaned, encoded, explored and used to compare models, and not only about the web page that shows the answer.

## 1.2 Problem Statement

Existing cost tools stop at an estimate. They do not tell a patient who cannot pay which NGO or welfare scheme can help, or by how much. As a result, eligible people never find the help that already exists. The problem addressed by this project is therefore twofold:

- to predict the medical cost for an Indian profile with good accuracy, using properly cleaned and explored data; and
- to turn that predicted cost and the user's profile into a ranked list of NGO and welfare options with the likely aid in ₹.

## 1.3 Scope of the Project

The scope of the project was fixed at the start so that the work stayed realistic for one semester.

**Table 1.1 Scope of the project**

| In scope | Out of scope |
|---|---|
| Cost prediction for Indian profiles, in ₹ | Real insurance claims or policy pricing |
| Rule-based matching to NGOs and welfare schemes | Live updates of scheme rules |
| Interactive dashboard with analytics | Medical advice or diagnosis |

## 1.4 Objectives

- **Objective 1.** Clean the data and run exploratory data analysis (EDA) to find the main cost drivers: age, BMI, smoking and city tier.
- **Objective 2.** Encode the features and compare Linear Regression, Random Forest and Gradient Boosting models using MAE, RMSE and RZ.
- **Objective 3.** Curate an NGO and welfare dataset and build a rule-based engine that matches users to eligible aid.
- **Objective 4.** Build a dashboard that shows the cost estimate, the matched aid in ₹ and visual analytics.

## 1.5 Importance of the Application

Low-income patients and their families gain the most from a tool like this. Without a link between cost and aid, eligible people never find the help that exists. A patient counsellor at a hospital help desk can enter a profile in under a minute and show the family both a realistic cost and the schemes worth applying to. Because the matcher is built from simple, readable rules, the family can also see why a scheme was or was not suggested, which builds trust in the result.

From an academic point of view, the project covers the full data science cycle on a problem that matters locally: data collection, cleaning, feature engineering, EDA, model selection, evaluation, testing and delivery.

## 1.6 Target Users

- Hospital help desks and patient counsellors.
- NGOs and trusts that screen applicants for aid.
- Government and insurance health portals that want to guide citizens.
- Families who want to plan their medical expenses.

## 1.7 Literature Survey and Research Gap

Four recent works were studied closely. Table 1.2 lists what each one did and what it left out.

**Table 1.2 Summary of the literature survey**

| S.No | Author(s) and Year | Method / Technique | Key Findings | Limitation / Gap |
|---|---|---|---|---|
| 1 | Orji et al., 2024 [1] | XGBoost, GBM and Random Forest with SHAP/ICE on 986 records | Tree ensembles predict well; smoking, BMI and age drive cost | Prediction only; no help for patients who cannot pay |
| 2 | IJRASET, 2023 [2] | Linear, Decision Tree, Gradient Boosting; Streamlit app | Boosting and tree models beat plain linear regression | One cost estimate; no aid guidance |
| 3 | Hong et al., 2025 [3] | Logistic Regression, Decision Tree, Random Forest on 1M+ emergency-department bills | Decision Tree flagged 87% of patients needing aid | Finds need, but does not match NGOs or schemes |
| 4 | Rajpoot et al., 2025 [4] | AI recommendation system for government schemes | Helps citizens discover suitable schemes | No link to predicted medical cost |

**Research gap.** None of the surveyed work joins cost prediction with eligibility-matched NGO and welfare aid. This project turns a predicted cost and a user profile into a ranked list of aid options, which answers Objectives 3 and 4. The key contribution is the link between an ML cost prediction and a transparent, rule-based match to NGO and government aid, using a self-curated eligibility dataset that no public source provides.

# CHAPTER 2

# System Requirements

The project works on tabular data of a few thousand rows, so it needs only an ordinary laptop. No graphics card or server is needed.

## 2.1 Hardware Requirements

**Table 2.1 Hardware requirements**

| Component | Requirement | Reason |
|---|---|---|
| Processor | Laptop or PC, Intel Core i5 or similar | Training the models takes under a minute |
| Memory | 8 GB RAM | Data and models fit easily in memory |
| Storage | 256 GB SSD | The datasets are small (a few hundred kilobytes) |
| GPU | Not needed | The data is tabular; no deep learning is used |

## 2.2 Software Requirements

**Table 2.2 Software requirements**

| Item | Details |
|---|---|
| Operating system | Windows 10/11 (the code also runs on Linux and macOS) |
| Language | Python 3.11 |
| Notebook / editors | Jupyter Notebook, VS Code, Google Colab |
| Libraries | pandas, NumPy, scikit-learn, matplotlib, joblib |
| Web framework | Flask (back end), HTML, CSS and JavaScript (front end) |
| Version control | Git and GitHub |
| Browser | Any modern browser (Chrome, Edge, Firefox) |

## 2.3 Development Tools and Environment

Data exploration and early model trials were done in Jupyter Notebook and Google Colab. The final scripts were written in VS Code as small Python modules (configuration, data generation, preprocessing, training, matching and the Flask application) so that each part can be tested on its own. The whole project is kept in a Git repository on GitHub. A requirements file lists the six Python packages needed to run it, and all random choices use the fixed seed 42 so that anyone can repeat the results.

The project folder is organised as follows.

## Project structure

```
project/
|-- app.py                    Flask application (web server and JSON API)
|-- requirements.txt          numpy, pandas, scikit-learn, joblib, flask, matplotlib
|-- src/
|   |-- config.py             constants: seed, column names, paths
|   |-- generate_data.py      builds the India-adapted dataset (2,500 rows)
|   |-- preprocessing.py      cleaning, outlier check, encoding pipeline
|   |-- train.py              model comparison, grid search, saving the best model
|   `-- matching.py           rule-based aid matching engine (rules R1 to R5)
|-- data/                     raw and clean CSV, ngo_welfare_schemes.csv
|-- outputs/                  best_model.joblib, metrics.json, figures, test log
|-- templates/index.html      dashboard page
|-- static/                   style.css, app.js
`-- tests/                    test_matching.py, test_pipeline.py, test_api.py
```

# CHAPTER 3

# Technology Stack

Only free and widely used tools were chosen. The stack is small on purpose: every part can be explained and reproduced on a student laptop.

## 3.1 Front-end (HTML, CSS, JavaScript)

The dashboard is a single web page written in plain **HTML**, styled with **CSS** and made interactive with **JavaScript**. No front-end framework is used. The page has three tabs. JavaScript reads the form, sends it to the server with the browser's `fetch` call as JSON, and draws the cost estimate, the list of matched schemes and the analytics bars from the JSON that comes back. Amounts are shown in Indian number format, for example ₹1,67,537.

## 3.2 Back-end (Python, Flask, scikit-learn)

The back end is written in **Python 3.11**. The data and model work uses these libraries:

- **pandas and NumPy** for loading, cleaning and transforming the tables.
- **scikit-learn** for the preprocessing pipeline (`ColumnTransformer`, `StandardScaler`, `OneHotEncoder`), the three regression models, `GridSearchCV`, cross-validation and the evaluation metrics.
- **matplotlib** for the EDA and result figures used in this report.
- **joblib** to save the trained model to a file and load it again when the server starts.
- **Flask**, a light web framework, to serve the page and expose a small JSON API.

## 3.3 Data Storage (CSV, JSON, joblib)

The project does not need a database server. The data is small and read-only while the application runs, so plain files are enough: the raw and clean datasets and the scheme directory are CSV files, the evaluation results are kept in one JSON file, and the trained pipeline is saved as a joblib file. This keeps the setup to a single command and makes the data easy to open in Excel and check.

## 3.4 Version Control (Git, GitHub)

Git is used for version control and GitHub for sharing the code among the three team members. The repository holds the source code, the self-prepared dataset, the scheme directory and the tests.

**Table 3.1 Summary of the technology stack**

| Layer | Technology | Used for |
|---|---|---|
| Presentation | HTML, CSS, JavaScript | Form, tabs, result cards, analytics bars |
| Application | Flask (Python) | Routes, validation, JSON responses |
| Logic | scikit-learn pipeline, matching module | Cost prediction and aid matching |
| Data | CSV, JSON, joblib files | Datasets, scheme directory, metrics, saved model |
| Tooling | Jupyter, Colab, VS Code, Git, GitHub | Development, training, sharing |

# CHAPTER 4

# System Architecture

## 4.1 High-level Architecture Diagram and Methodology

The system works in two connected streams that meet in a matching engine, as shown in Fig 4.1. In the first stream the medical cost dataset is cleaned, encoded and scaled, explored, and used to train and compare the three models; the best one is saved. In the second stream the NGO and welfare dataset is turned into a set of eligibility rules. When a user submits a profile, the saved model predicts the cost, and the matching engine compares the profile and the predicted cost with every scheme. The result goes to the dashboard.

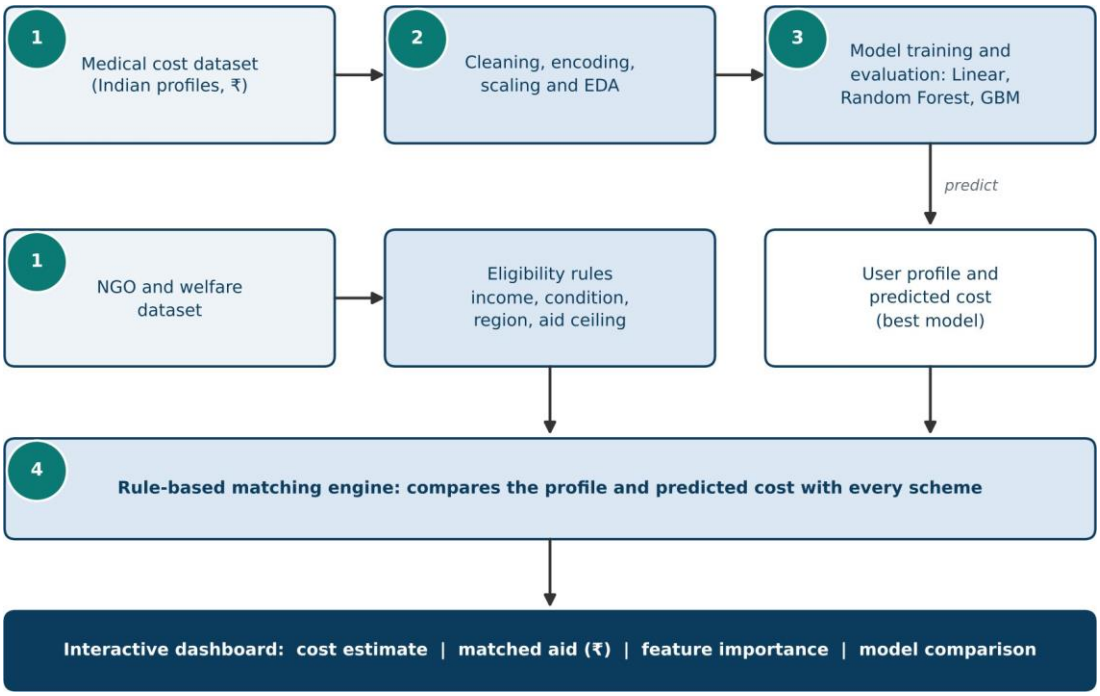



Fig No. 4.1 High-level architecture of the system

The work followed five steps:

**Table 4.1 Methodology followed**

| Step | Name | What was done |
|---|---|---|
| 1 | Define the problem | Cost estimates alone do not help patients who cannot afford care. |
| 2 | Collect and clean | Medical cost data (₹) and NGO/welfare data were prepared; the data was cleaned, encoded and scaled. |
| 3 | EDA and models | The data was studied, then Linear, Random Forest and Gradient Boosting models were compared. |
| 4 | Match and build | The rule-based aid matching and the web dashboard were built. |
| 5 | Test and evaluate | MAE, RMSE, RZ, cross-validation and 25 test cases were run. |

## 4.2 Description of Components

The software is arranged in four layers (Fig 4.2). Each layer talks only to the layer next to it.

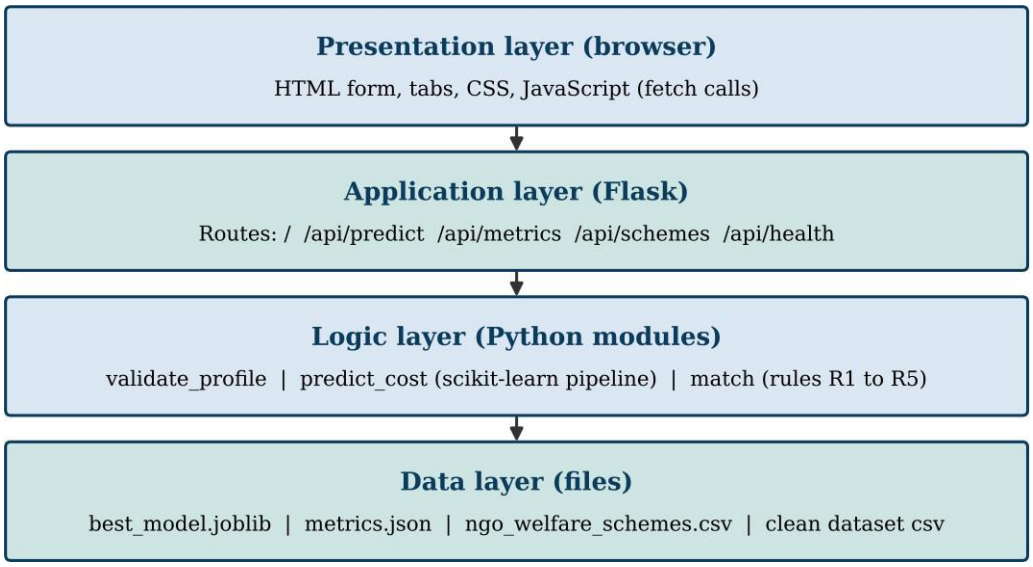



Fig No. 4.2 Layered architecture of the application

- **Data layer.** The medical cost dataset (2,500 records, nine features, amounts in ₹) and the NGO/welfare directory (twelve entries). It also holds the saved model and the metrics file.
- **Preprocessing and EDA.** Cleaning, encoding, scaling and the plots. This code runs when the model is trained and the same pipeline is stored inside the saved model, so a new profile receives exactly the same treatment as the training data.
- **Prediction models.** Linear Regression, Random Forest Regression and Gradient Boosting Regression, compared on the same split. The best one is used by the web application.
- **Matching engine and dashboard.** Five rules on income, condition and region produce the aid in ₹; the Flask application and the browser page present the cost, the aid and the analytics.

The matching engine follows the decision flow in Fig 4.3 for every scheme in the directory.

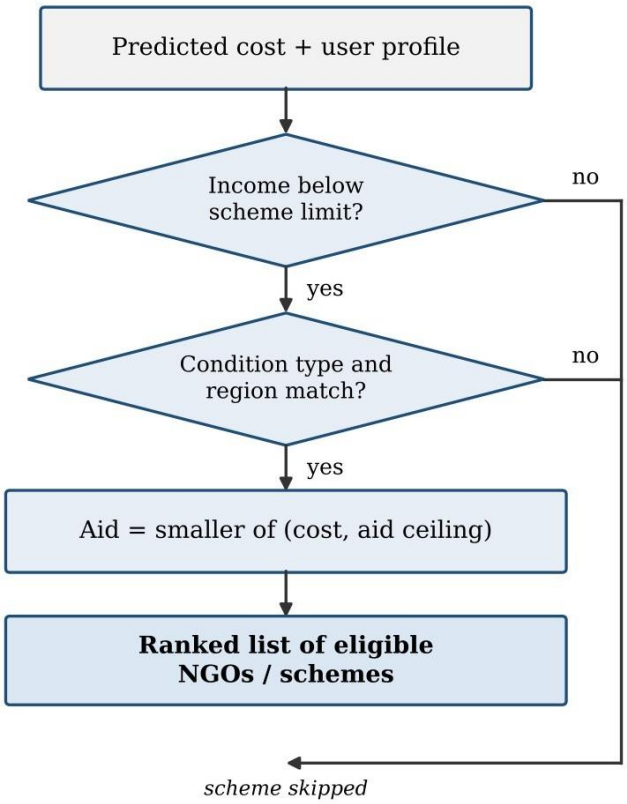



Fig No. 4.3 Decision flow of the matching engine for one scheme

## 4.3 Deployment Architecture

The application runs on a single machine. Models are trained on a laptop or on Google Colab. The Flask server loads the saved model and the scheme directory once when it starts and then answers requests from the browser on the same machine. No cloud service, container or CI/CD pipeline is used, which keeps the system easy to run in a lab or at a help desk. The steps are given in Chapter 9.

# CHAPTER 5

# Design

## 5.1 Wireframes or Mockups

The interface was planned as one page with three tabs so that a counsellor never has to leave the screen. The layout was sketched first and then built in HTML and CSS. The final pages are shown in Fig 5.1 to Fig 5.3.

- **Predict and Find Aid.** A form on the left (age, BMI, children, smoker, city tier, region, annual income and condition type) and the results on the right: a large cost estimate in ₹ and below it a numbered list of matched schemes, each with the provider type, rules, documents needed and the likely aid.
- **Analytics.** A model comparison table, RZ bars for the three models, and the feature importance of the best model.
- **Schemes Directory.** A table of all schemes in the directory with their income limit, condition, region and aid ceiling.

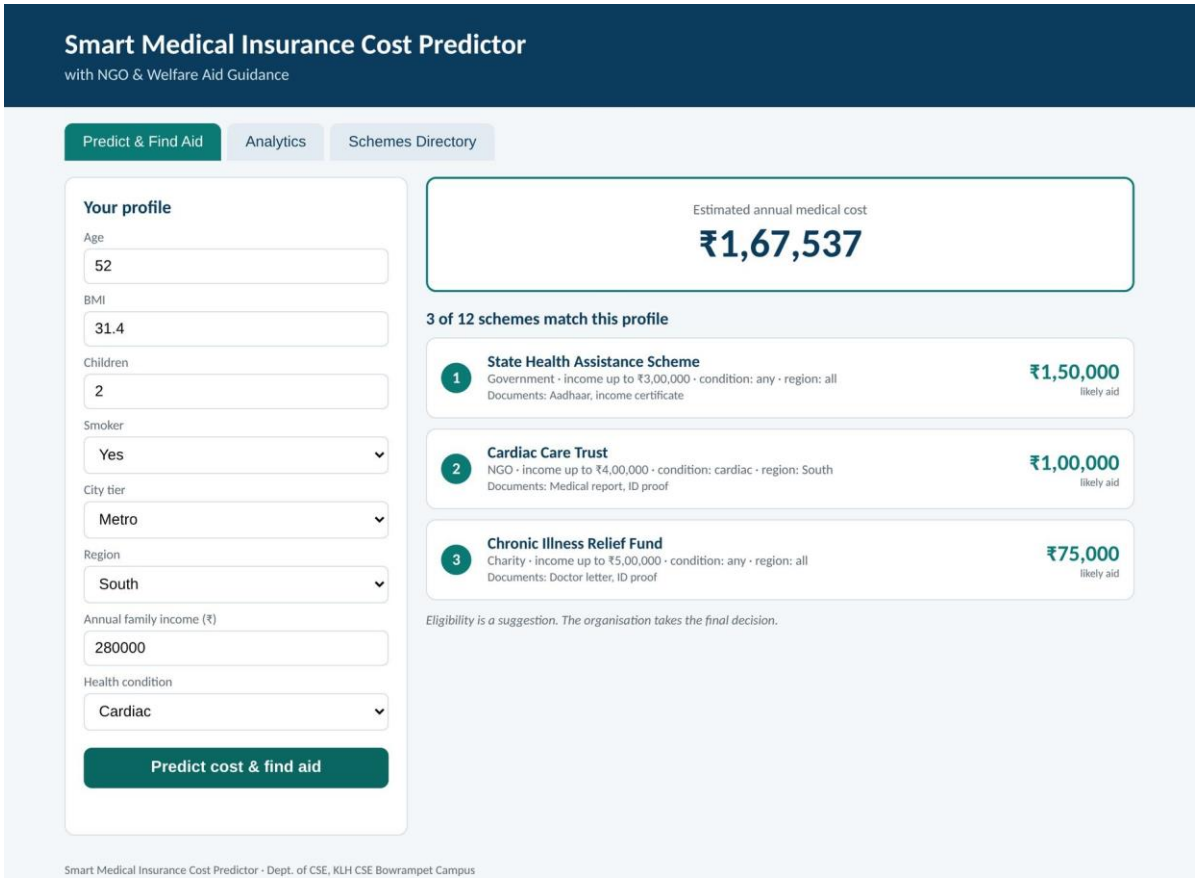



Fig No. 5.1 Predict and Find Aid tab with a cost estimate and matched schemes

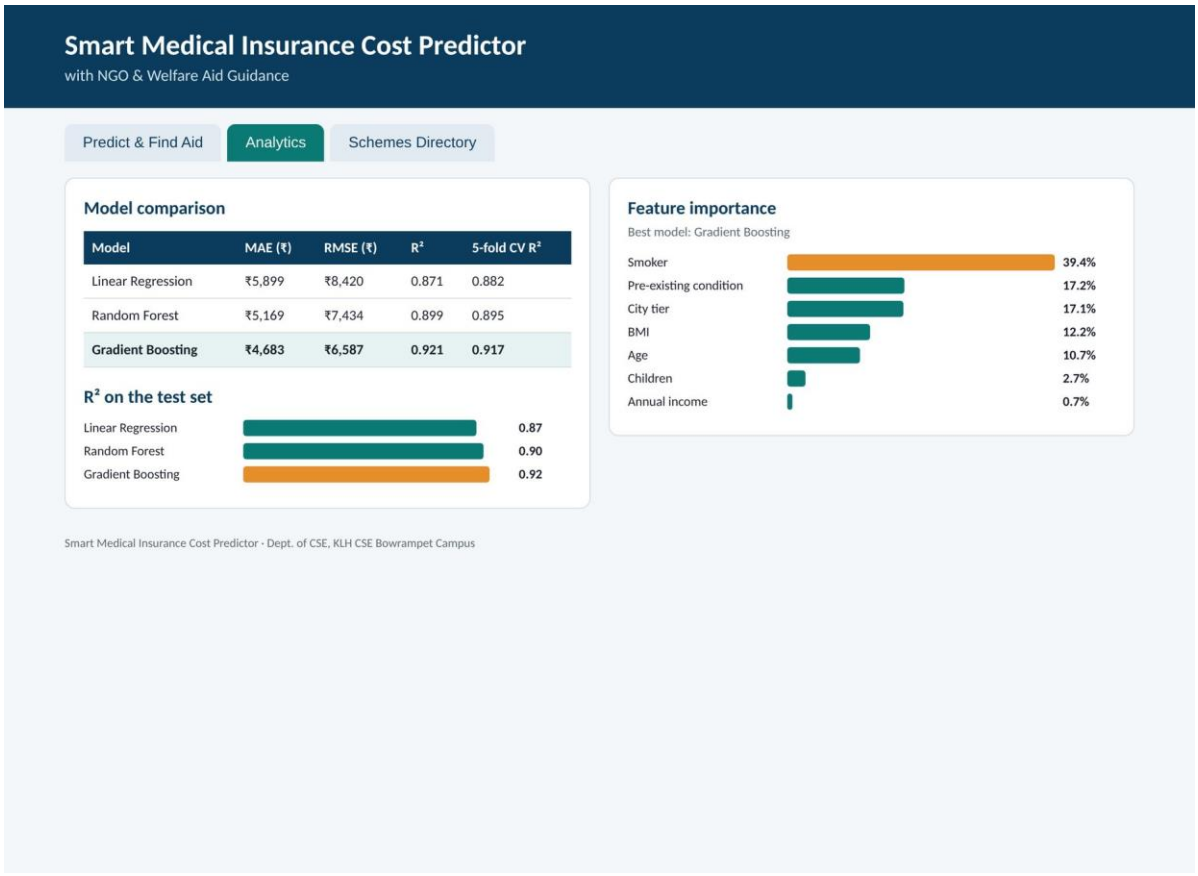



Fig No. 5.2 Analytics tab: model comparison and feature importance

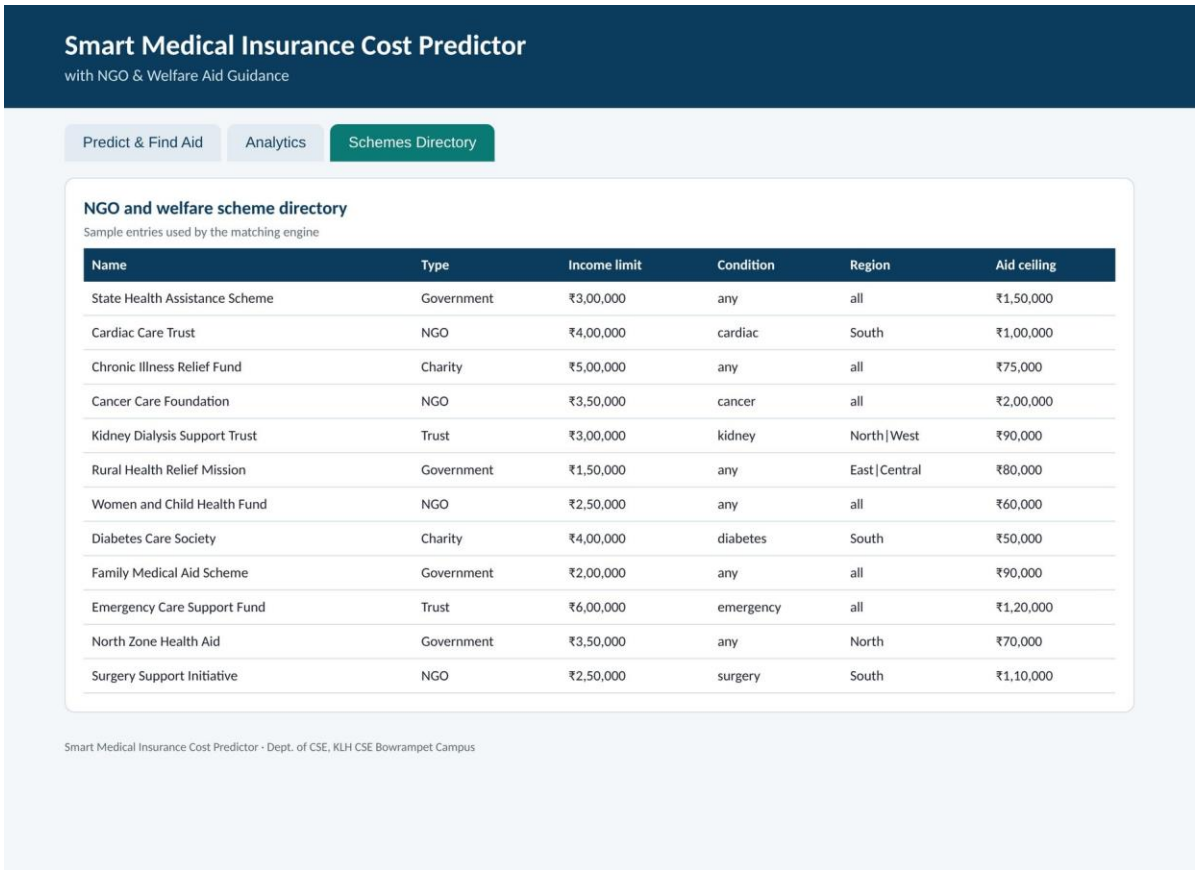



Fig No. 5.3 Schemes Directory tab

## 5.2 Data Schema (ER-style Diagram)

There are two stored tables and one transient input. The **medical_cost** table holds the 2,500 training records. The **schemes** table holds the aid directory. The **user profile** is not stored; it exists only while a request is processed and is matched against the schemes table (Fig 5.4).

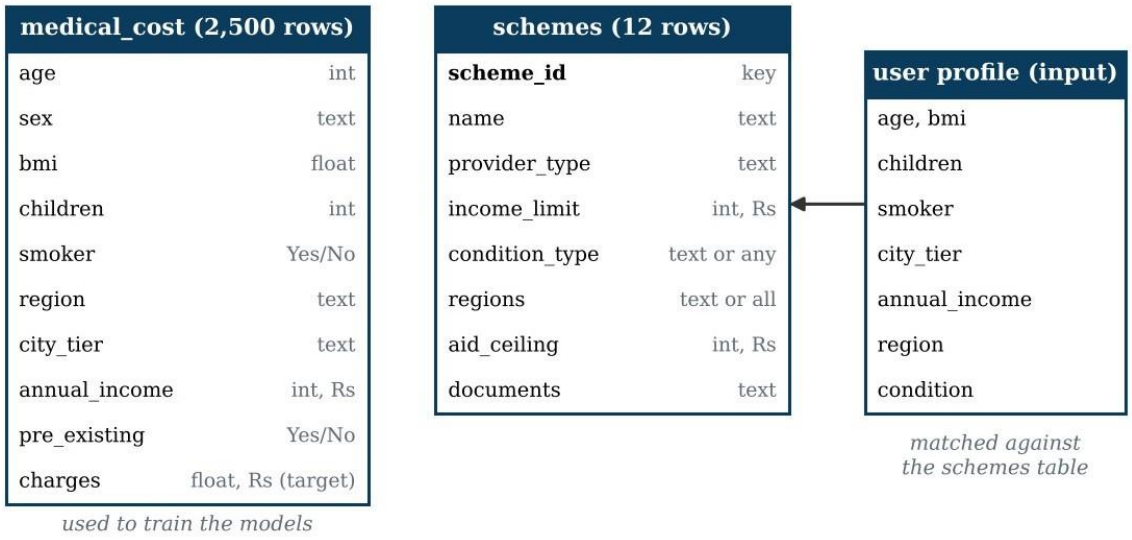



Fig No. 5.4 Data schema of the project

**Table 5.1 Columns of the medical cost dataset**

| Column | Type | Meaning |
|---|---|---|
| age | integer | Age in years (18 to 64) |
| sex | text | Male or Female |
| bmi | float | Body mass index |
| children | integer | Number of children (0 to 4) |
| smoker | Yes / No | Smoking status |
| region | text | North, South, East, West or Central |
| city_tier | text | Metro, Tier-2 or Tier-3 |
| annual_income | integer | Yearly family income in ₹ |
| pre_existing | Yes / No | Pre-existing medical condition |
| charges | float | Yearly medical cost in ₹ (target) |

**Table 5.2 Columns of the NGO and welfare scheme directory**

| Column | Meaning |
|---|---|
| scheme_id | Unique key, S01 to S12 |
| name, provider_type | Name of the scheme and whether it is Government, NGO, Trust or Charity |
| income_limit | Highest annual income (₹) allowed; the limit itself is allowed |
| condition_type | A condition such as cardiac, cancer, kidney, diabetes, emergency, surgery, or "any" |
| regions | One or more regions separated by "\|", or "all" |
| aid_ceiling | Largest amount of aid (₹) the scheme gives |

| Column | Meaning |
|---|---|
| documents | Documents the applicant should carry |
| source, date_checked | Where the entry came from and when it was last checked |

## 5.3 Data Flow Diagrams

The level-0 diagram (Fig 5.5) shows the whole system as one process. The user sends a profile and receives a cost and an aid list. The medical cost dataset and the scheme directory feed the system.

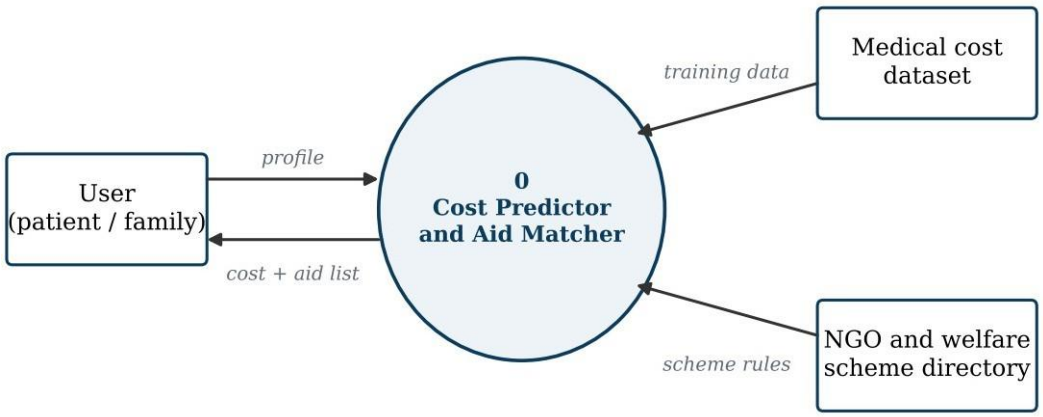



Fig No. 5.5 Data flow diagram, level 0

The level-1 diagram (Fig 5.6) opens the process into five steps. Raw data is cleaned into clean data (1.0), which is used to train and evaluate the models (2.0); the best model is saved as a model file. For each request the profile is used to predict the cost (3.0), the cost and the scheme list go to the matching step (4.0), and the dashboard (5.0) returns the estimate and the aid list to the user.

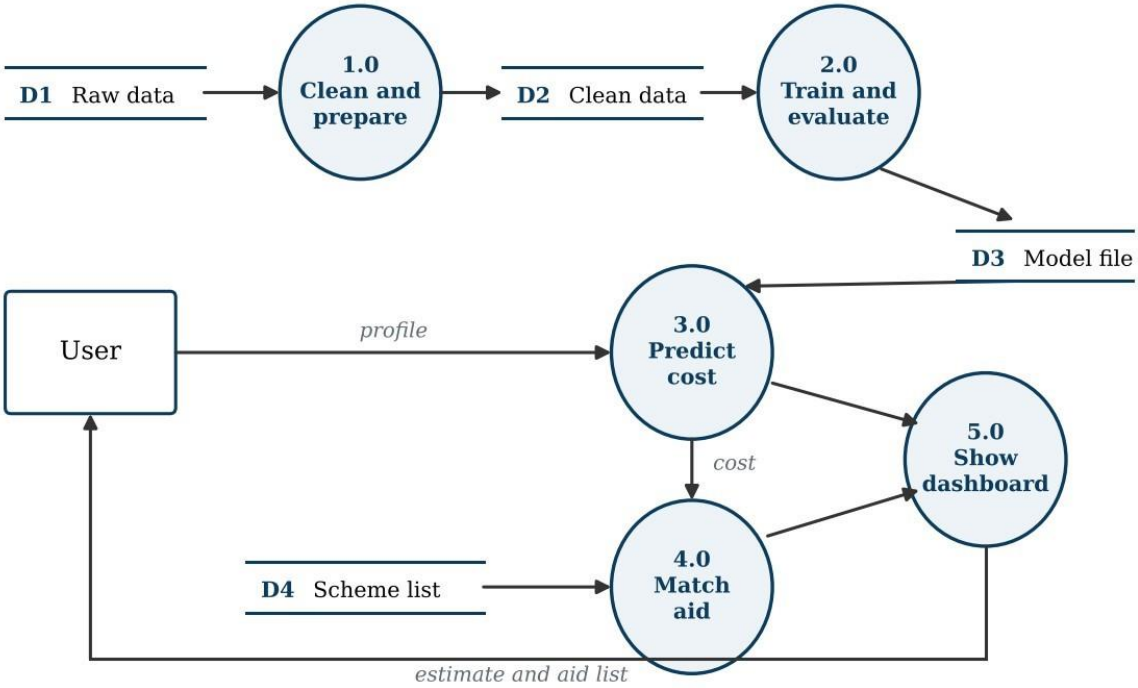



Fig No. 5.6 Data flow diagram, level 1

# CHAPTER 6
# Implementation

## 6.1 Module-wise Implementation

The project is divided into four modules, listed in Table 6.1. Sections 6.1.1 to 6.1.4 describe how each one was built.

**Table 6.1 Modules of the project**

| No. | Module | Purpose | Input → Output | Developed by |
|---|---|---|---|---|
| 1 | Data Preprocessing and EDA | Clean, encode and explore the cost dataset | Raw records → clean data and plots | S. Trividha |
| 2 | Cost Prediction Models | Train and compare three regression models | Clean data → best model and metrics | K. Deepthi |
| 3 | NGO Dataset and Matching Engine | Curate aid data and apply eligibility rules | Profile + cost → ranked aid list (₹) | M. Swetha |
| 4 | Dashboard and Integration | Combine prediction, matching and charts | User form → cost, aid and analytics | All members |

### 6.1.1 Module 1: Data Preprocessing and Exploratory Analysis

#### (a) The dataset

Public medical-cost datasets are mostly from other countries and carry no Indian city tier, region or income. The team therefore prepared an India-adapted dataset with a Python script (`generate_data.py`). It creates 2,500 records, fixed by the random seed 42, with nine features and the yearly medical cost in ₹. The feature values follow realistic Indian ranges: age 18 to 64, BMI centred near 25.6, about 13% smokers, 22% with a pre-existing condition, five regions (North, South, East, West, Central), three city tiers and a right-skewed family income centred near ₹3.2 lakh. The cost is built from a base amount that grows with age, smoking (more again when BMI is above 30), BMI above 25, number of children and a pre-existing condition; it is then scaled by city tier (Metro 1.25, Tier-2 1.0, Tier-3 0.85) and given about 9% random variation. These relationships follow the patterns reported in the literature [1], [5], [6]. The dataset is self-prepared, so the costs are a rough guide and not a bill.

**Table 6.2 Dataset summary**

| Property | Value |
|---|---|
| Records (rows) | 2,500 |
| Features (columns) | 9 inputs and 1 target (charges, ₹) |
| Missing values | 75 in BMI (3%) |
| Duplicates | 0 |
| Mean / median charges | ₹57,486 / ₹52,650 |
| Standard deviation of charges | ₹23,562 |

| Property | Value |
|---|---|
| Minimum / maximum charges | ₹17,690 / ₹2,15,330 |
| Smokers / pre-existing condition | 12.7% / 21.0% |
| Records per city tier | Tier-2 949, Tier-3 819, Metro 732 |
| Records per region | South 616, North 609, West 579, East 440, Central 256 |

| age | sex | bmi | children | smoker | region | city_tier | annual_income | pre_existing | charges |
|---|---|---|---|---|---|---|---|---|---|
| 42 | Female | 26.9 | 4 | No | Central | Tier-2 | 111000 | No | 59600.0 |
| 26 | Male | 25.8 | 1 | No | North | Tier-2 | 250000 | Yes | 62910.0 |
| 47 | Male | 28.8 | 0 | No | East | Tier-2 | 786000 | Yes | 94610.0 |
| 49 | Male | 29.9 | 1 | No | West | Tier-3 | 745000 | No | 49620.0 |
| 18 | Male | 29.8 | 0 | No | North | Tier-3 | 621000 | No | 33340.0 |
| 22 | Male | 25.1 | 1 | No | East | Metro | 386000 | Yes | 61060.0 |
| 40 | Female | 32.0 | 0 | No | West | Tier-3 | 486000 | No | 50820.0 |
| 34 | Male | 26.7 | 3 | No | West | Tier-2 | 343000 | Yes | 65410.0 |

Fig No. 6.1 First rows of the dataset

**(b) Data cleaning**

- **Missing values.** 75 BMI values (3%) were missing (Fig 6.2). They were filled with the median BMI of 25.6, which is not pulled by a few very high values. The imputation is repeated for every new record in the same way.
- **Duplicates.** The duplicate check found none, so no row was removed (2,500 rows in, 2,500 rows out).
- **Outliers.** The IQR rule (values outside Q1 − 1.5×IQR and Q3 + 1.5×IQR) marked 13 BMI values (outside 13.5 to 37.5) and 88 charges above ₹1,10,329. The unusual BMI values are medically possible. The high charges belong to the patients who need help most, so **all outliers were kept**.

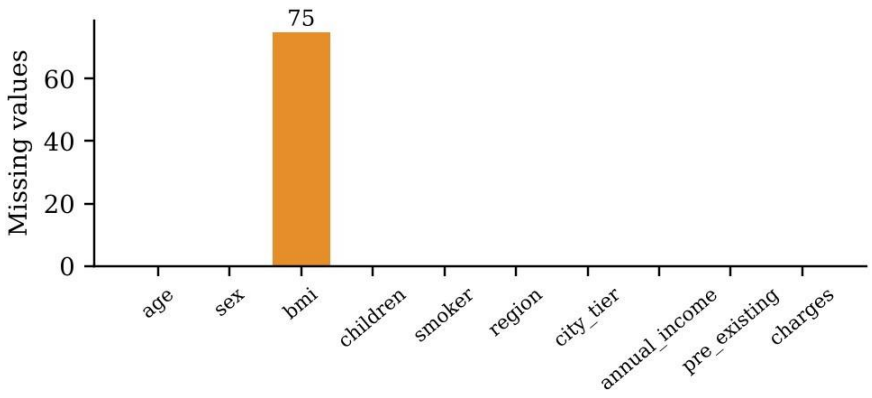



Fig No. 6.2 Missing values per column before cleaning

**(c) Exploratory data analysis**

The target is right-skewed: the mean (₹57,486) is higher than the median (₹52,650) and the skewness is 1.47 (Fig 6.3, left). A log transform reduces the skewness to 0.22 (Fig 6.3, right). Because the final answer must be a rupee amount, and tree models do not need a symmetric target, the log transform was tested but the raw target was kept for the final models. In the test, the log target raised the RZ of Linear Regression from 0.871 to 0.888.

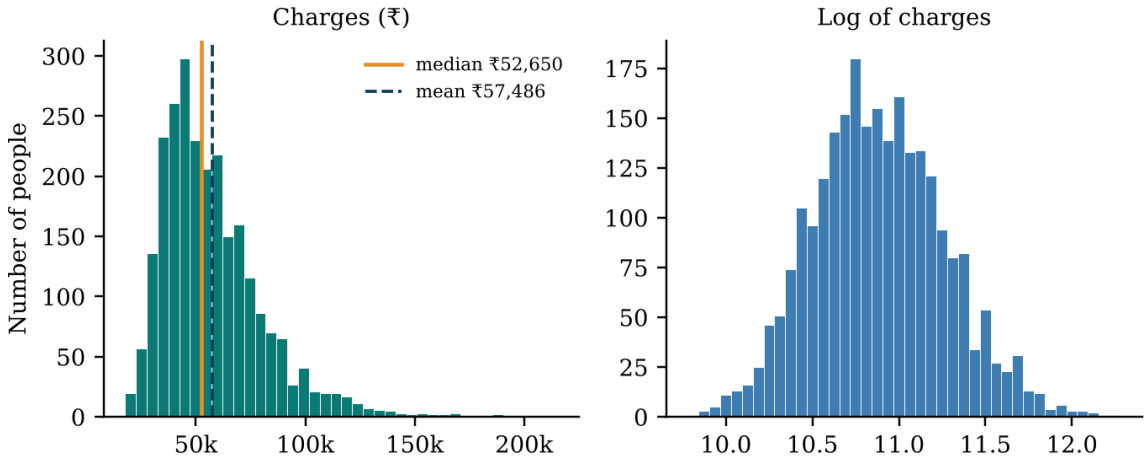



Fig No. 6.3 Distribution of charges and of log of charges

The box plots in Fig 6.4 compare costs across groups. Smokers cost far more than non-smokers (average ₹94,639 against ₹52,072). Costs fall with the city tier: Metro ₹70,788, Tier-2 ₹56,439 and Tier-3 ₹46,811. People with a pre-existing condition average ₹76,718 against ₹52,387 for those without.

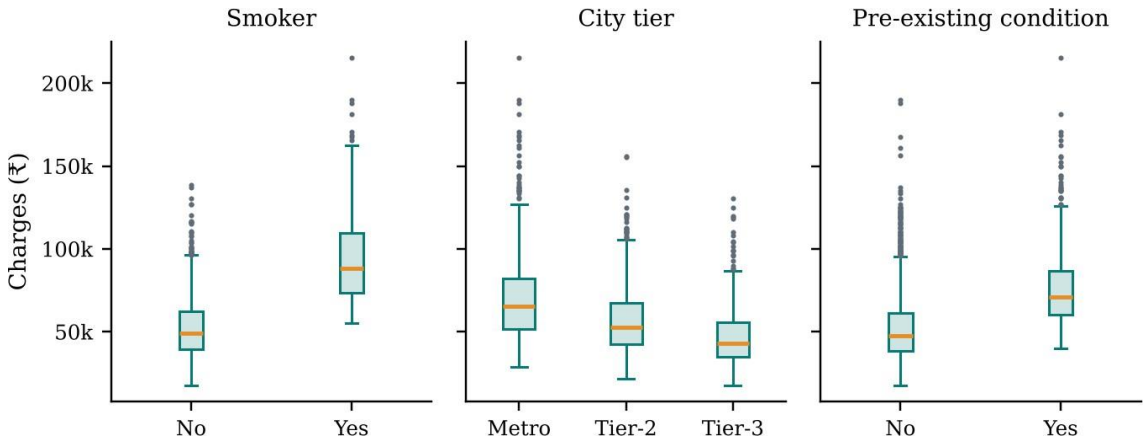



Fig No. 6.4 Charges by smoking status, city tier and pre-existing condition

Fig 6.5 shows that charges rise with age and with BMI, and that smokers (orange) sit well above non-smokers (green) at every age and BMI.

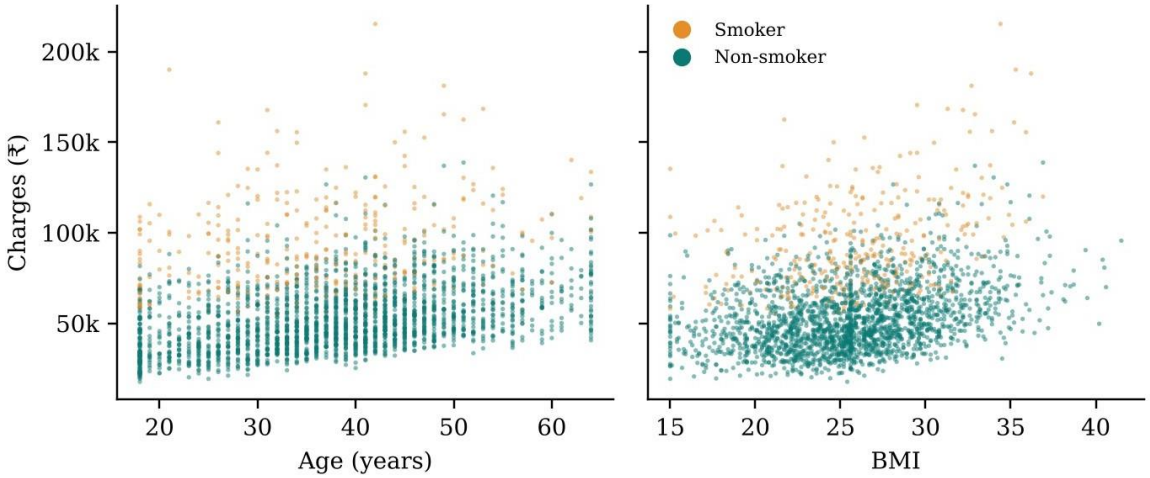



Fig No. 6.5 Charges against age and against BMI, coloured by smoking status

The correlation heat map (Fig 6.6) gives the same picture in numbers. Charges correlate most with smoking (0.60), then pre-existing condition (0.42), BMI (0.30), age (0.28) and children (0.16). Income has almost no link with charges (0.02). The features are almost uncorrelated with each other, so there is no multicollinearity problem for Linear Regression.

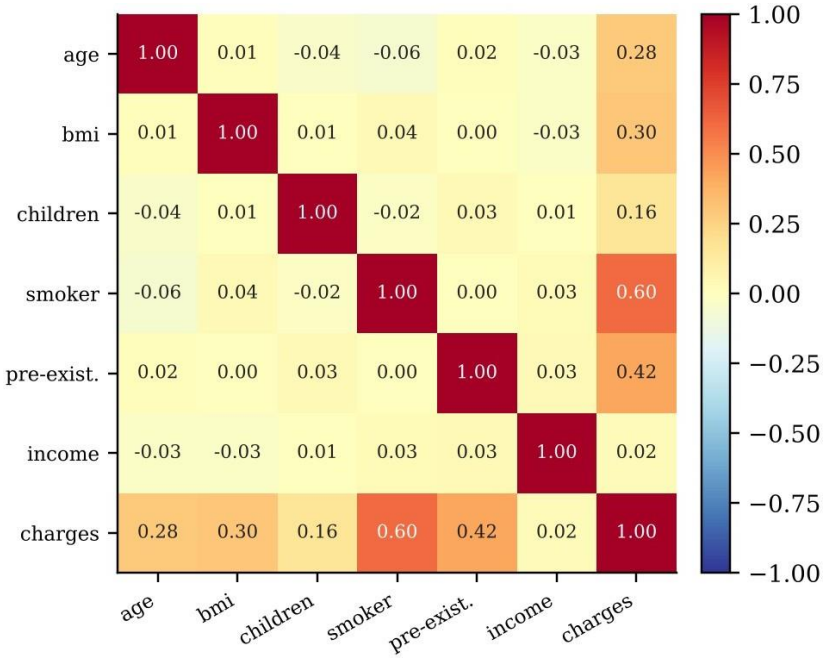



Fig No. 6.6 Correlation heat map of the numeric and binary features

**(d) Transformation and feature selection**

- **Encoding.** Binary columns (smoker, pre-existing condition) are coded 0/1 and the city tier is one-hot encoded, using `OneHotEncoder(drop="first")` so that no column is redundant.
- **Scaling.** The numeric columns (age, BMI, children, annual income) are standardised with `StandardScaler`.

- **One pipeline.** Imputation, encoding and scaling sit inside a single scikit-learn `Pipeline` together with the model, so the same steps are applied to training data, test data and a new user profile, and no information leaks from the test set.
- **Feature selection.** A Gradient Boosting model on all nine columns gave sex an importance of only 0.09% and region 0.02%. Dropping them changed RZ from 0.921 to 0.918. Seven of the nine features were therefore kept: age, BMI, children, smoker, city tier, income and pre-existing condition. After encoding, the seven features become eight model inputs.

**Code 6.1 Preprocessing pipeline (src/preprocessing.py)**

```
def clean(df):
    """Remove duplicates, fill missing BMI with the median, report outliers.
    High charges are kept on purpose: they are the patients who need help most."""
    report = {"rows_in": int(len(df))}
    report["missing_before"] = {c: int(v) for c, v in df.isna().sum().items() if v}
    df = df.drop_duplicates().reset_index(drop=True)
    report["duplicates_removed"] = report["rows_in"] - int(len(df))
    median_bmi = float(df["bmi"].median())
    df = df.copy()
    df["bmi"] = df["bmi"].fillna(median_bmi)
    report["bmi_median_used"] = median_bmi
    report["missing_after"] = int(df.isna().sum().sum())
    report["outliers"] = {}
    for col in ("bmi", "charges"):
        n, lo, hi = iqr_outliers(df[col])
        report["outliers"][col] = {"count": n, "low": lo, "high": hi}
    report["rows_out"] = int(len(df))
    return df, report

def build_preprocessor(num_cols=NUM_COLS, cat_cols=CAT_COLS):
    return ColumnTransformer([
        ("num", StandardScaler(), list(num_cols)),
        ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), list(cat_cols)),
    ])
```

### 6.1.2 Module 2: Cost Prediction Models

Three regression models were chosen to go from a simple baseline to a stronger one (Table 6.3).

**Table 6.3 Algorithms used**

| Model | Why chosen | Key parameters |
|---|---|---|
| Linear Regression | Simple, fast baseline that is easy to explain | Scaled numeric inputs; one-hot encoded categories |
| Random Forest Regression [9] | Handles non-linear effects and interactions; stable | 300 trees, max depth 10, min leaf 3 |
| Gradient Boosting Regression [10] | Learns from earlier errors; strong on tabular data | 300 trees, learning rate 0.05, depth 3 |

**Data splitting.** The data was split 80% for training (2,000 rows) and 20% for testing (500 rows) with `random_state=42`. The test rows were not used for any choice. Five-fold cross-validation was run on the training part.

**Selection criteria.** The best model has the lowest MAE and RMSE and the highest RZ on the test set, and it is checked with cross-validation. The three metrics are:

- **MAE** (mean absolute error) is the average size of the error in ₹.
- **RMSE** (root mean squared error) is also in ₹ but punishes large errors more.
- **$R^2$** is the share of the variation in cost that the model explains; adjusted RZ corrects it for the number of inputs.

**Grid search.** A grid search with 5-fold cross-validation and MAE as the score was run for Gradient Boosting over the number of trees (100, 300), learning rate (0.05, 0.1) and depth (2, 3, 4), which is twelve combinations (Table 6.4). The best cross-validated MAE was ₹4,772, for 100 trees, rate 0.1 and depth 3. The setting used for the final model, 300 trees, rate 0.05 and depth 3, scored ₹4,812, less than 1% worse. A slower learning rate with more trees is usually steadier on new data, so this setting was kept for the models.

**Table 6.4 Six best Gradient Boosting settings in the grid search (5-fold CV MAE)**

| Rank | Trees | Learning rate | Depth | CV MAE (₹) |
|---|---|---|---|---|
| 1 | 100 | 0.1 | 3 | ₹4,772 |
| 2 | 300 | 0.05 | 3 | ₹4,812 |
| 3 | 300 | 0.1 | 3 | ₹4,824 |
| 4 | 300 | 0.1 | 2 | ₹4,830 |
| 5 | 300 | 0.05 | 2 | ₹4,850 |
| 6 | 300 | 0.05 | 4 | ₹4,879 |

**Code 6.2 Model settings and pipeline (src/train.py)**

```
def make(name):
    if name == "Linear Regression": return LinearRegression()
    if name == "Random Forest":
        return RandomForestRegressor(n_estimators=300, max_depth=10,
                                     min_samples_leaf=3, random_state=42)
    return GradientBoostingRegressor(n_estimators=300, learning_rate=0.05,
                                     max_depth=3, random_state=42)

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
pipe = Pipeline([("pre", build_preprocessor()), ("model", make(name))])
pipe.fit(Xtr, ytr)
cv_r2 = cross_val_score(pipe, Xtr, ytr, cv=5, scoring="r2").mean()
```

### 6.1.3 Module 3: NGO Dataset and Matching Engine

**The scheme directory.** Because no public source lists aid rules in a common format, the team built the directory by hand (`ngo_welfare_schemes.csv`). Every entry is put in one standard rule format: income limit, condition type, region list, aid ceiling and documents, plus a source note and the date it was checked. The twelve entries are **sample entries** made to test and demonstrate the engine; they are modelled on the kinds of government schemes, trusts and charities that exist and must be checked against the real organisation before anyone relies on them.

**Table 6.5 The scheme directory used by the matching engine**

| ID | Scheme | Type | Income limit (₹) | Condition | Region | Aid ceiling (₹) |
|---|---|---|---|---|---|---|
| S01 | State Health Assistance Scheme | Government | 3,00,000 | any | all | 1,50,000 |
| S02 | Cardiac Care Trust | NGO | 4,00,000 | cardiac | South | 1,00,000 |
| S03 | Chronic Illness Relief Fund | Charity | 5,00,000 | any | all | 75,000 |
| S04 | Cancer Care Foundation | NGO | 3,50,000 | cancer | all | 2,00,000 |
| S05 | Kidney Dialysis Support Trust | Trust | 3,00,000 | kidney | North, West | 90,000 |
| S06 | Rural Health Relief Mission | Government | 1,50,000 | any | East, Central | 80,000 |
| S07 | Women and Child Health Fund | NGO | 2,50,000 | any | all | 60,000 |
| S08 | Diabetes Care Society | Charity | 4,00,000 | diabetes | South | 50,000 |
| S09 | Family Medical Aid Scheme | Government | 2,00,000 | any | all | 90,000 |
| S10 | Emergency Care Support Fund | Trust | 6,00,000 | emergency | all | 1,20,000 |
| S11 | North Zone Health Aid | Government | 3,50,000 | any | North | 70,000 |
| S12 | Surgery Support Initiative | NGO | 2,50,000 | surgery | South | 1,10,000 |

**The rules.** The matching engine is rule-based on purpose, so that every result can be explained to the patient. It applies five rules to each scheme:

**Table 6.6 Rules of the matching engine**

| Rule | Meaning |
|---|---|
| R1 | The user's annual income is less than or equal to the scheme's income limit (the limit is inclusive). |
| R2 | The scheme's condition is "any" or equals the user's condition. |
| R3 | The scheme's region is "all" or the user's region is in its region list. |
| R4 | Likely aid = the smaller of the predicted cost and the scheme's aid ceiling. |
| R5 | Eligible schemes are sorted by likely aid, from high to low. |

Input is checked before matching. Age must be 18 to 100, BMI 10 to 60, children 0 to 10, income cannot be negative or empty, and smoker, city tier, region and condition must be one of the allowed values. Otherwise the user gets a clear error message and nothing is computed.

**Code 6.3 The matching function (src/matching.py)**

```
def match(profile, predicted_cost, schemes):
    """Return eligible schemes with the likely aid, best first."""
    out = []
    for _, s in schemes.iterrows():
        if profile["annual_income"] > s["income_limit"]:                    # R1
            continue
        if s["condition_type"] != "any" and s["condition_type"] != profile["condition"]:    # R2
            continue
        if s["regions"] != "all" and profile["region"] not in s["regions"].split("|"):      # R3
            continue
        aid = min(float(predicted_cost), float(s["aid_ceiling"]))           # R4
        out.append({"scheme_id": s["scheme_id"], "name": s["name"], "provider_type":
s["provider_type"],
                    "condition_type": s["condition_type"], "regions": s["regions"],
                    "income_limit": int(s["income_limit"]), "aid_ceiling": int(s["aid_ceiling"]),
                    "likely_aid": round(aid), "documents": s["documents"]})
    out.sort(key=lambda r: (-r["likely_aid"], r["name"]))                    # R5
    return out
```

### 6.1.4 Module 4: Dashboard and Integration

The fourth module joins the others. The Flask application loads the saved model, the scheme directory and the metrics once at start-up. When the browser submits a profile, the application validates it, predicts the cost, runs the matching engine and returns one JSON answer. The same page also shows the analytics and the scheme directory. All three members built this module together. Sections 6.2 to 6.5 give the details.

## 6.2 Front-end Logic (UI Rendering, State Handling)

The page keeps no hidden state. Three small pieces of JavaScript do all the work:

- **Tabs.** A click on a tab button hides all panels and shows the chosen one.
- **Form submit.** The form values are read into an object and posted as JSON to `/api/predict`. If the server answers with an error, the message is shown under the form and any old result is cleared, so a stale estimate is never left on the screen (Fig 6.7). Otherwise the estimate and one card per matched scheme are drawn.
- **Analytics and directory.** When the page loads, `/api/metrics` and `/api/schemes` are fetched once and the comparison table, the RZ bars, the feature-importance bars (sorted by value) and the directory table are filled in.

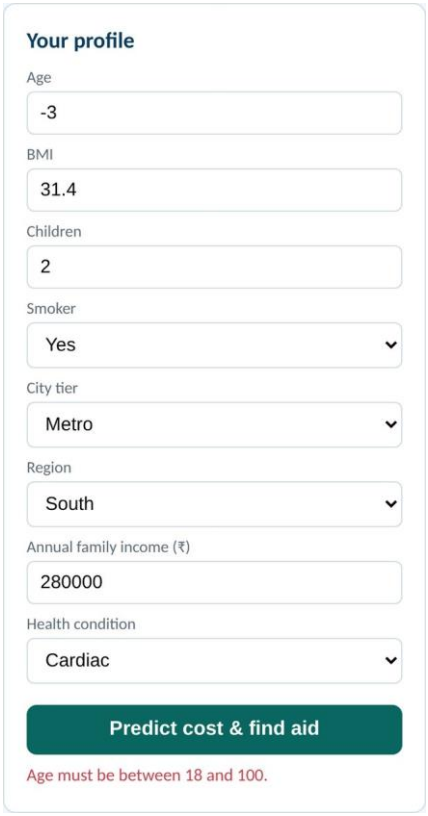



Fig No. 6.7 Validation error shown for an invalid input

## 6.3 Back-end Logic (Routes and Services)

The Flask application has one page route and four API routes (Table 6.7). The logic itself sits in the `src` package, so the routes stay short.

**Table 6.7 Routes of the application**

| Route | Method | Purpose |
|---|---|---|
| / | GET | Serves the dashboard page |
| /api/health | GET | Returns status, number of schemes and the model name |
| /api/predict | POST | Validates a profile, predicts the cost and returns the matched schemes |
| /api/metrics | GET | Returns model comparison, best model, feature importance and parameters |
| /api/schemes | GET | Returns the full scheme directory |

**Code 6.4 The prediction route (app.py)**

```
def predict_cost(profile):
    row = pd.DataFrame([{k: profile[k] for k in C.FEATURES}])
    return float(max(MODEL.predict(row)[0], 0))

@app.post("/api/predict")
def api_predict():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return jsonify(error="Send the profile as JSON."), 400
    try:
        profile = validate_profile(data)
    except ValueError as e:
        return jsonify(error=str(e)), 400
    cost = predict_cost(profile)
    matches = match(profile, cost, SCHEMES)
    return jsonify(estimate=round(cost), matches=matches, matched=len(matches),
total_schemes=int(len(SCHEMES)))
```

## 6.4 Data Loading and Model Persistence

After training, the best pipeline (preprocessing and model together) is written to `outputs/best_model.joblib` and the evaluation results to `outputs/metrics.json`. The cleaned dataset is saved as a CSV. When the server starts it reads these files once with `joblib.load`, `json.load` and `pandas.read_csv`, so a request only has to call `predict`. This keeps an answer fast: a single prediction takes about 3 milliseconds and a full request to `/api/predict` about 7 milliseconds on the development laptop.

The final deployed model is Gradient Boosting trained on the seven selected features with the settings in Table 6.3.

## 6.5 API Integration

The front end and back end talk only through the internal JSON API of Table 6.7. A request to `/api/predict` and its answer look like this:

**Code 6.5 Example request and response**

```
POST /api/predict
{"age": 52, "bmi": 31.4, "children": 2, "smoker": "Yes", "city_tier": "Metro",
 "region": "South", "annual_income": 280000, "condition": "cardiac"}

200 OK
{"estimate": 167537, "matched": 3, "total_schemes": 12,
 "matches": [{"scheme_id": "S01", "name": "State Health Assistance Scheme", "likely_aid":
150000, ...},
             {"scheme_id": "S02", "name": "Cardiac Care Trust", "likely_aid": 100000, ...},
             {"scheme_id": "S03", "name": "Chronic Illness Relief Fund", "likely_aid":
75000, ...}]}

400 Bad Request   {"error": "Age must be between 18 and 100."}
```

# CHAPTER 7

# Features

## 7.1 List of Main Features

- Cost estimate in ₹ for an Indian profile from the best-performing model.
- Rule-based matching to NGO, trust, charity and government schemes with the likely aid in ₹ and the documents needed.
- Ranked results, with the largest likely aid first.
- Clear validation messages for wrong or missing input.
- Analytics tab with the model comparison and the feature importance.
- Schemes Directory tab that shows every scheme and its rules.
- A note on every result reminding the user that eligibility is a suggestion and the organisation decides.

## 7.2 How Each Feature Works (User and Technical View)

### 7.2.1 Cost estimate and aid matching

**User view.** The user opens the Predict and Find Aid tab, fills in the profile and presses the button. The yearly cost estimate appears at once and below it the list of schemes. For the example profile (a 52-year-old male smoker with BMI 31.4, two children, living in a Metro city in the South, income ₹2,80,000, with a cardiac condition) the system shows an estimate of ₹1,67,537 and three matching schemes (Fig 7.1).

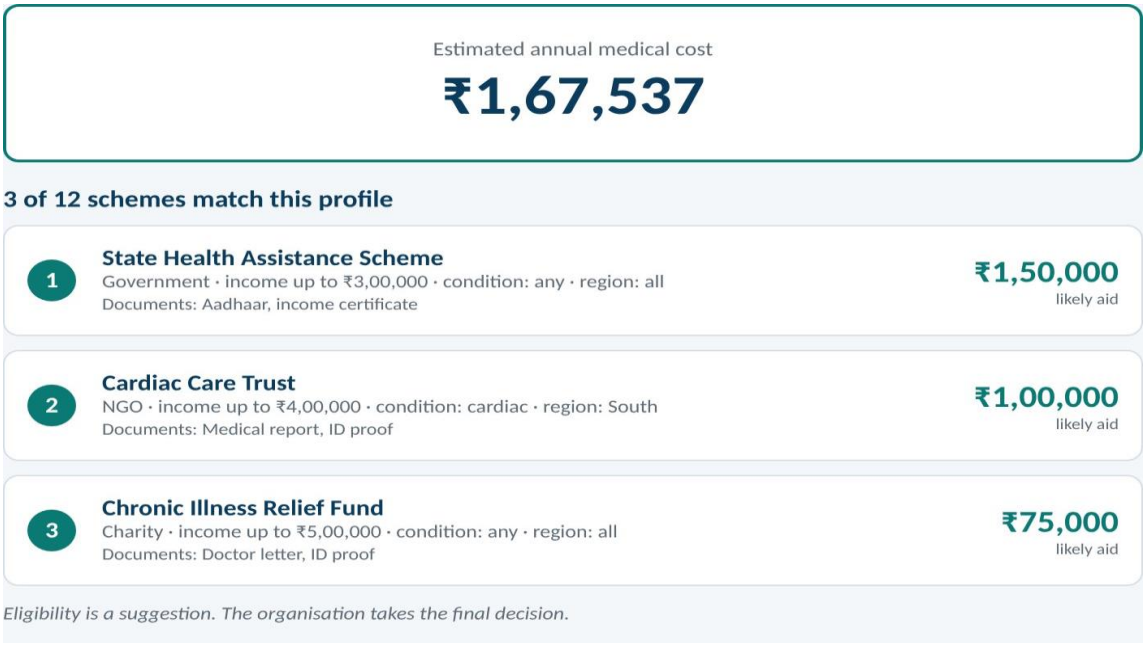



Fig No. 7.1 Estimate and matched schemes for the example profile

**Technical view.** The form is sent as JSON, validated, converted to a one-row table with the seven model inputs, and passed to the saved pipeline, which scales and encodes it and predicts the cost. The cost and the profile then go through rules R1 to R5 (Table 6.6). In the example, S01 (income ≤ ₹3,00,000, any condition, all regions) passes; S02 (≤ ₹4,00,000, cardiac, South) passes; S03 (≤ ₹5,00,000, any, all) passes. The other nine fail on income, condition or region. The likely aid is the smaller of the cost and each ceiling: ₹1,50,000, ₹1,00,000 and ₹75,000.

**Table 7.1 Example profile and result**

| Field | Value |
|---|---|
| Age / BMI / children | 52 / 31.4 / 2 |
| Smoker | Yes |
| City tier / region | Metro / South |
| Annual income | ₹2,80,000 |
| Condition | Cardiac |
| Estimated yearly cost | ₹1,67,537 |
| Matched schemes | S01 ₹1,50,000; S02 ₹1,00,000; S03 ₹75,000 |

A second profile shows the other end of the range. A 29-year-old non-smoker with BMI 23.4, no children, in a Tier-3 city of the East, with income ₹1,20,000 and no condition, gets an estimate of ₹25,787 and five matches (Fig 7.2). Here the cost is below every ceiling, so the likely aid for each scheme equals the cost.

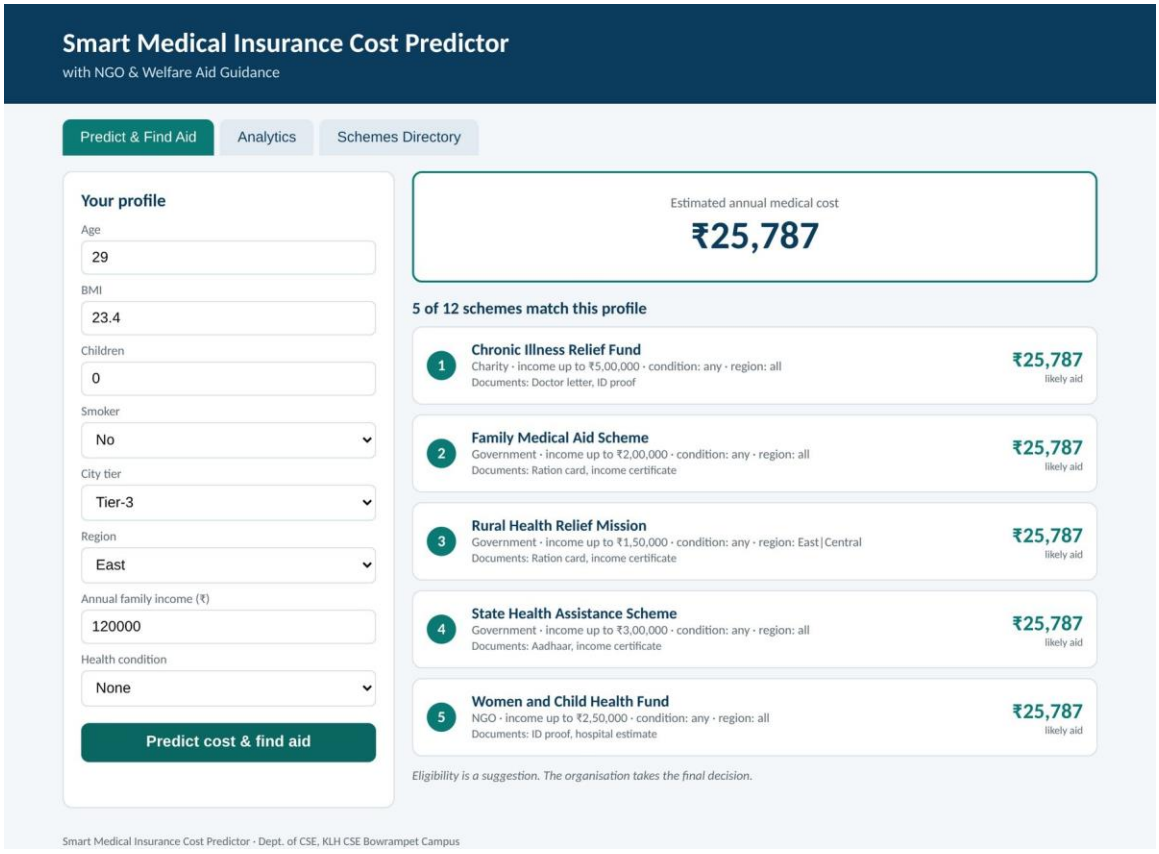



Fig No. 7.2 Result for a low-cost, low-income profile

### 7.2.2 Input validation

If the user enters, for example, an age of −5 or leaves the income empty, the server answers with HTTP 400 and a plain message and the page shows it under the form (Fig 7.3). No exception reaches the user.

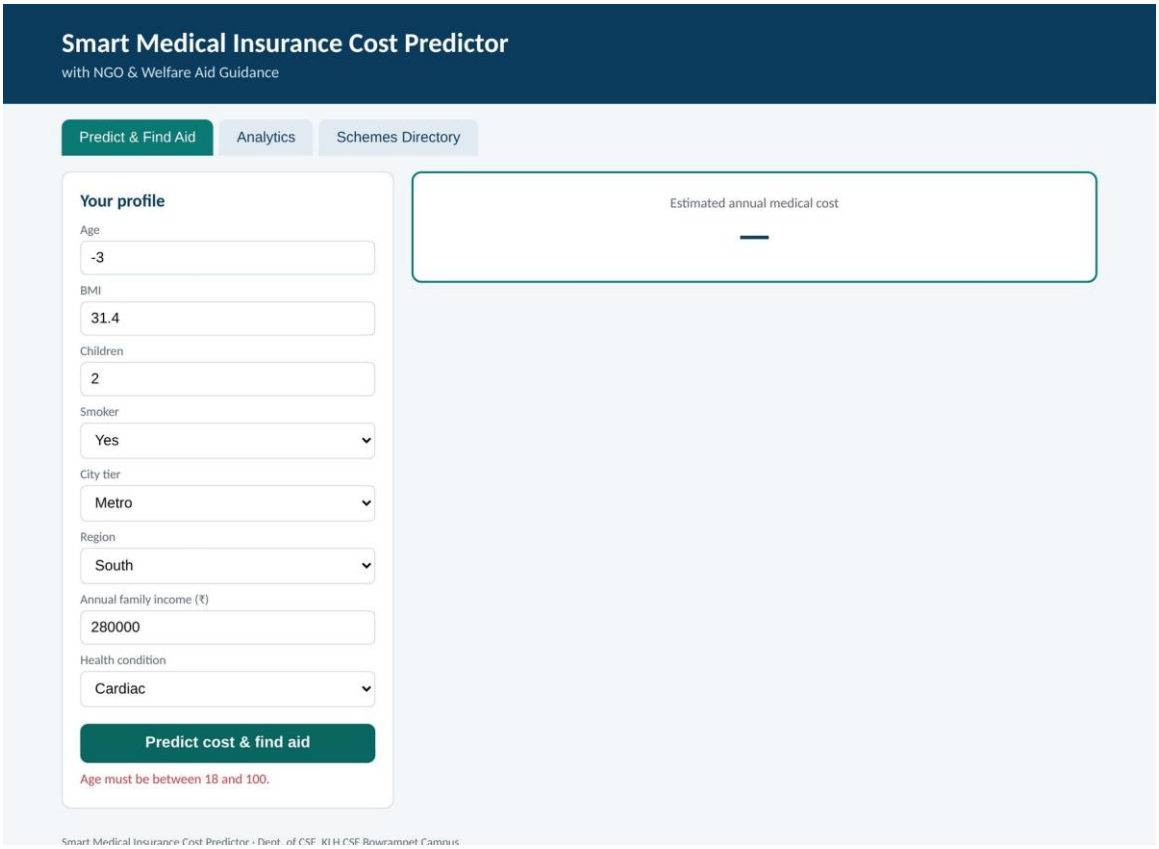



Fig No. 7.3 Validation message for an invalid age

### 7.1.1 Analytics

The Analytics tab (Fig 5.2) shows the model comparison table and the feature-importance bars, so a viewer can see why the Gradient Boosting model was chosen and which inputs matter most. The figures come from the metrics file written during training, so they always match the saved model.

### 7.1.2 Schemes directory

The Schemes Directory tab (Fig 5.3) lists all twelve entries with their rules. A counsellor can use it to explain why a family did not match a scheme, for example because their income is above the limit.

# CHAPTER 8
# Testing

## 8.1 Functional and API Testing

Each matching rule has its own unit test, and the web API is tested end to end with the Flask test client. The tests check that the page loads, that a valid profile returns an estimate and three matches, and that a bad age, an empty income or a body that is not JSON returns HTTP 400 with a message instead of crashing. Data-cleaning tests check that no missing values remain, that no rows are lost, and that the high charges are kept.

## 8.2 Integration Testing

The API tests also act as integration tests, because one request passes through validation, the saved scikit-learn pipeline and the matching engine. The model tests load the real saved model and score it on the same 500 test rows. A separate test checks a business sense rule: with every other input equal, a smoker must be predicted to cost more than a non-smoker.

## 8.3 Tools Used for Testing

- **unittest**, the Python standard test library, to write and run all 25 test cases.
- **Flask test client** to call the routes without starting a server.
- **Hand-worked examples.** The matcher was checked against answers worked out by hand, such as the example profile in Chapter 7.
- **Hold-out set and 5-fold cross-validation** to validate the models.

**Command to run all tests**

```
python -m unittest discover -s tests -t . -v
```

## 8.4 Test Cases and Results

### 8.4.1 Model evaluation results

All models were compared on the same 500 test rows after training on the 2,000 training rows (Table 8.1 and Fig 8.1). The cross-validation score comes from five folds of the training data.

**Table 8.1 Performance comparison on the test set (all nine features)**

| Metric | Linear Regression | Random Forest | Gradient Boosting |
|---|---|---|---|
| MAE (₹) | ₹5,899 | ₹5,169 | ₹4,683 |
| RMSE (₹) | ₹8,420 | ₹7,434 | ₹6,587 |
| RZ | 0.871 | 0.899 | 0.921 |

| Adjusted RZ | 0.867 | 0.897 | 0.919 |
|---|---|---|---|
| **Metric** | **Linear Regression** | **Random Forest** | **Gradient Boosting** |
| 5-fold CV RZ | 0.882 | 0.895 | 0.917 |

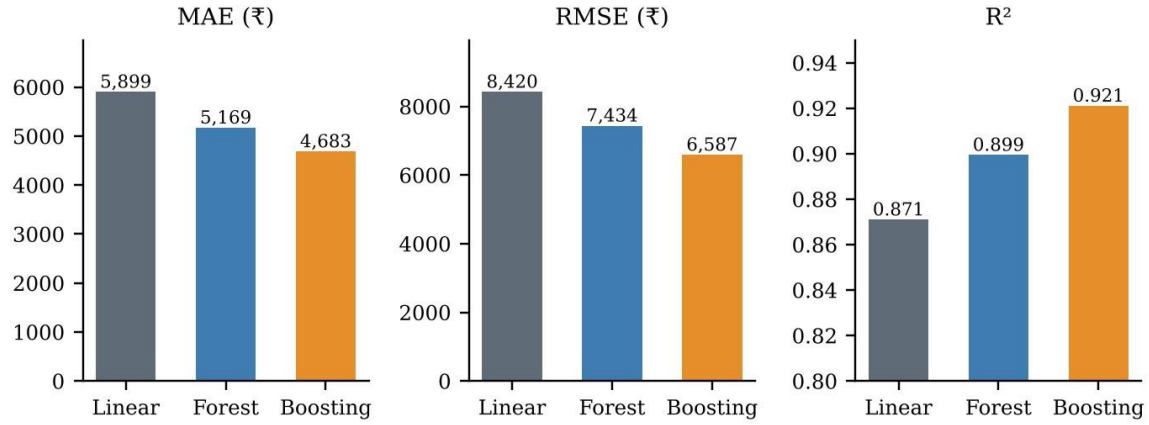



Fig No. 8.1 MAE, RMSE and RZ of the three models

Gradient Boosting is best on every measure: RZ = 0.921 and MAE = ₹4,683, ahead of Random Forest (0.899) and Linear Regression (0.871). The cross-validation scores (0.917, 0.895 and 0.882) agree with the test scores, so the result is not luck of the split.

The final deployed model uses only the seven selected features. Table 8.2 shows that dropping sex and region costs almost nothing in accuracy.

**Table 8.2 Seven-feature models (used by the application)**

| Metric | Linear Regression | Random Forest | Gradient Boosting |
|---|---|---|---|
| MAE (₹) | ₹5,903 | ₹5,159 | ₹4,717 |
| RMSE (₹) | ₹8,413 | ₹7,414 | ₹6,701 |
| RZ | 0.871 | 0.900 | 0.918 |
| Adjusted RZ | 0.869 | 0.898 | 0.917 |
| 5-fold CV RZ | 0.882 | 0.898 | 0.916 |

### 8.4.2 Analysis and key findings

**Interpretation.** Gradient Boosting explains about 92% of the variation in cost. Fig 8.2 shows predictions lying close to the diagonal line, and the errors in the right panel are centred on zero. About 66% of test predictions are within ₹5,000 of the actual cost and 90% are within ₹10,000. The median error is ₹3,582.

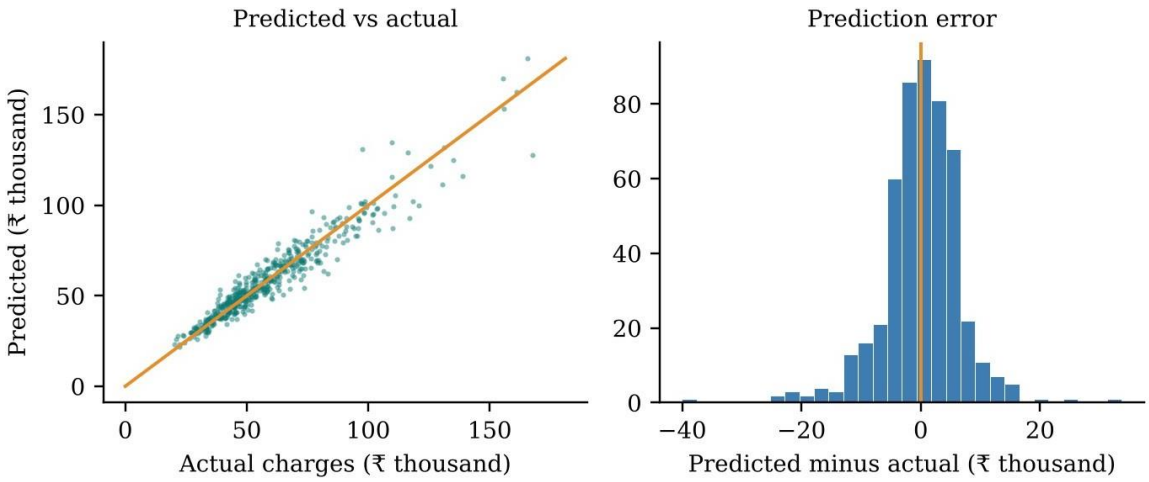



Fig No. 8.2 Predicted against actual charges and the prediction error

**Error analysis.** The largest errors are for rare, very high-cost patients with few training examples. For the 16 test patients with charges above ₹1,10,329 the mean error is ₹13,818, while for everyone else it is ₹4,381. The largest single error is about ₹40,000.

**Key drivers.** Fig 8.3 shows the importance of each input in the final model. Smoking status is the strongest driver (39%), followed by pre-existing condition (17%), city tier (17%), BMI (12%) and age (11%). Number of children (3%) and income (0.7%) add little; income matters for the aid rules, not for the cost. This agrees with the EDA and with earlier studies [1], [2].

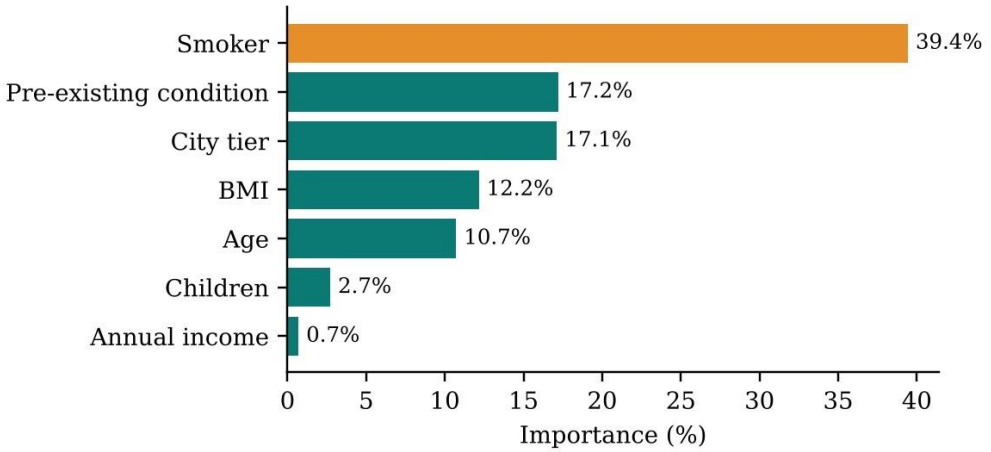



Fig No. 8.3 Feature importance of the Gradient Boosting model

### 8.4.3 Test cases

Twenty-five test cases were written, and all of them passed ("Ran 25 tests ... OK"). Table 8.3 lists them.

**Table 8.3 Test cases and results**

| Test ID | Test Description | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC-01 | Income equal to scheme limit | Scheme shown (limit inclusive) | Scheme shown | Pass |
| TC-02 | Income above scheme limit | Scheme skipped | Scheme skipped | Pass |

| Test ID | Test Description | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC-03 | Scheme with condition “any”, region “all” | Shown for every profile | Shown for all test profiles | Pass |
| TC-04 | Region not in scheme's region list | Scheme skipped | Scheme skipped | Pass |
| TC-05 | Condition different from scheme's condition | Scheme skipped | Scheme skipped | Pass |
| TC-06 | Cost larger or smaller than the ceiling | Aid = smaller of cost and ceiling | Aid = smaller value | Pass |
| TC-07 | Several schemes match | Sorted by aid, high to low | Sorted correctly | Pass |
| TC-08 | Example profile against the directory | 3 schemes matched | 3 schemes matched | Pass |
| TC-09 | Negative age | Clear error, no crash | Error message shown | Pass |
| TC-10 | Empty income | Clear error, no crash | Error message shown | Pass |
| TC-11 | Unknown region | Clear error, no crash | Error message shown | Pass |
| TC-12 | Missing values after cleaning | None left | 0 missing | Pass |
| TC-13 | Rows after cleaning; missing BMI found | 2,500 rows kept; 75 found | 2,500 rows; 75 found | Pass |
| TC-14 | High charges after cleaning | Maximum charge kept | Kept | Pass |
| TC-15 | Shape of encoded training data | 2,000 rows, 8 inputs | (2000, 8) | Pass |
| TC-16 | RZ of the saved model on test rows | Above 0.90 | 0.918 | Pass |
| TC-17 | Predictions on test rows | All positive | All positive | Pass |
| TC-18 | Smoker against non-smoker, other inputs equal | Smoker costs more | Smoker costs more | Pass |
| TC-19 | Home page request | Page loads (HTTP 200) | Loaded | Pass |
| TC-20 | Health endpoint | Status ok, 12 schemes | Status ok, 12 schemes | Pass |
| TC-21 | Predict for the example profile | Estimate and 3 matches | Estimate and 3 matches | Pass |
| TC-22 | Predict with a bad age | HTTP 400 with message | 400 with message | Pass |
| TC-23 | Predict with empty income | HTTP 400 | 400 | Pass |
| TC-24 | Body that is not JSON | HTTP 400 | 400 | Pass |
| TC-25 | Metrics and schemes endpoints | Data returned | Data returned | Pass |

**Validation method.** The models were validated with a 20% hold-out set and 5-fold cross-validation; the matcher was checked against hand-worked answers. **Performance.** A prediction takes about 3 ms and a complete request to the API about 7 ms, so the answer appears in well under a second.

# CHAPTER 9

# Deployment

## 9.1 Steps to Run and Deploy

The application is deployed on a local machine. The steps are:

**Table 9.1 Steps to run the application**

| Step | Action | Command |
|---|---|---|
| 1 | Get the code from the GitHub repository | git clone <repository-url> |
| 2 | Create and activate a virtual environment (optional) | python -m venv venv |
| 3 | Install the libraries | pip install -r requirements.txt |
| 4 | Generate the data and train the models | python -m src.train |
| 5 | Run the tests | python -m unittest discover -s tests -t . |
| 6 | Start the web application | python app.py |
| 7 | Open the dashboard in a browser | http://127.0.0.1:5000 |

Step 4 creates the raw and clean datasets if they are missing, runs the cleaning and the model comparison, and writes the saved model and the metrics file used by the web application.

## 9.2 Environment Configuration

- Python 3.11 with the packages in `requirements.txt` (numpy, pandas, scikit-learn, joblib, flask, matplotlib).
- All settings (random seed 42, number of records, column names, file paths) are in `src/config.py`, so nothing is hard-coded elsewhere.
- The server listens on 127.0.0.1, port 5000, with debug mode switched off.
- No passwords, keys or personal data are used or stored; the user's profile is processed in memory and then discarded.

## 9.3 Hosting

Training can be done on the local machine or on Google Colab. The dashboard runs on the local machine, which is enough for a demonstration, a lab or a help desk with one computer. Moving it to a public server would need a production web server and HTTPS, and is listed under future enhancements (Chapter 11).

# CHAPTER 10
# Challenges & Limitations

## 10.1 Issues Faced During Development

- **Skewed, very high charges.** A few patients have much higher costs than the rest.
- **Different rule formats across NGOs.** Every organisation describes its eligibility differently.
- **Missing BMI values.** About 3% of the records had no BMI.
- **Little public Indian cost data.** No open dataset has Indian city tier, region and income with cost.
- **Dashboard bugs found while testing.** The feature-importance bars first overflowed because the server returned the values sorted by name; an old result stayed on screen after an error; and one scheme, first called a senior-citizen scheme, matched a 29-year-old because the engine has no age rule.

## 10.2 Solutions Applied

**Table 10.1 Problems and solutions**

| Problem | Solution |
|---|---|
| Skewed, very high charges | Outliers were kept (they are the patients who need help) and a log transform was tested |
| Different rule formats across NGOs | One standard rule format with source and date for every entry |
| Missing BMI values | Median imputation inside the pipeline |
| Little public Indian cost data | An India-adapted dataset in ₹ was built with a documented script and a fixed seed |
| Bars out of order | The page now sorts the importance values before drawing |
| Stale result after an error | The page clears the old result as soon as an error comes back |
| Scheme matched an unsuitable age | The entry was renamed Family Medical Aid Scheme; age rules are left for future work |

## 10.3 Current Limitations

- The dataset is self-prepared, so costs are a rough guide and not a bill.
- There are only 2,500 records, so the models can overfit.
- The NGO data is collected by hand, the twelve entries are samples, and the directory needs regular updates.
- Eligibility is a suggestion; the organisation decides.

- The matching engine uses only income, condition and region; it has no age, gender or document check.
- The dashboard runs locally and has no user accounts.

# CHAPTER 11

# Future Enhancements

## 11.1 Planned Features

- Tune the models further, try XGBoost and add SHAP explanations that show why a cost was predicted.
- Use real Indian claims data and cover more illnesses and states.
- Add age and gender rules and a document checklist to the matcher.
- Build a public web app with Hindi and Telugu, plus a mobile version.

## 11.2 Possible Integrations and Optimizations

- Cloud hosting, with no storage of personal data.
- Regular, verified updates of scheme rules, for example through a data-entry page for NGOs.
- Integration with hospital help-desk software and with government and insurance health portals.

# CHAPTER 12

# Conclusion

## 12.1 Summary of the Project

The project built a cost predictor and an aid matcher on an India-adapted medical cost dataset in ₹ and delivered both in one dashboard. The data was cleaned and explored, seven of nine features were kept, and three models were compared. The aid matcher uses five transparent rules over a directory of NGO and welfare schemes.

## 12.2 What Was Achieved

- **Gradient Boosting was best**, with RZ = 0.921 and MAE = ₹4,683, ahead of Random Forest (0.899) and Linear Regression (0.871).
- **All four objectives were met:** cleaning and EDA, model comparison, an NGO dataset with rule-based matching, and the dashboard.
- **Smoking, pre-existing condition, city tier, BMI and age** were found to be the main cost drivers.
- **All 25 test cases passed.**

**Table 12.1 Team contribution**

| Reg. No. | Name | Contribution / Modules handled |
|---|---|---|
| 2420030010 | S. Trividha | Data collection, preprocessing and EDA (Module 1); report writing |
| 2420030484 | K. Deepthi | Model building and evaluation (Module 2); results and analysis |
| 2420030116 | M. Swetha | NGO and welfare dataset, rule-based matching engine (Module 3) |

The dashboard and integration (Module 4) were built together by all members.

## 12.3 Skills Learned

- Cleaning real-world style data: missing values, outliers and skew.
- Exploratory data analysis and reading plots and correlations.
- Building a leak-free preprocessing pipeline and comparing models with proper metrics and cross-validation.
- Designing a transparent rule engine and writing automated tests for it.
- Building a small Flask application with a JSON API and a browser front end.

- Working as a team with Git and GitHub, and reporting results honestly, including the limits of the data.

# CHAPTER 13

# References

## A. Papers and articles


[1] Orji et al., "Machine learning for an explainable cost prediction of medical insurance," arXiv, 2024.

[2] "Medical insurance cost prediction using machine learning," Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET), 2023.

[3] Hong et al., "Predicting a need for financial assistance in emergency department care," PMC/NCBI, 2025.

[4] M. Rajpoot, A. Thakur, and S. Kumar, "Government schemes AI recommendation system," 2025.

[5] U. S. Sabarinath and A. Mathew, "Medical insurance cost prediction," IJDCN, vol. 4, no. 4, pp. 1–4, 2024.

[6] D. S. AbdElminaam et al., "An efficient framework for predict medical insurance costs using machine learning," J. Comput. Commun., vol. 3, no. 2, pp. 55–64, 2024.

[7] P. Mohanraj et al., "AI enabled system for simplified access to government schemes for all citizens," IJEETR, vol. 8, no. 2, pp. 4365–4375, 2026.

[8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[9] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[10] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.


## B. Documentation sites used


- scikit-learn documentation: https://scikit-learn.org
- Flask documentation: https://flask.palletsprojects.com
- pandas documentation: https://pandas.pydata.org

# CHAPTER 14

# Appendices

## A. Screenshots of the Application

These screens were captured from the running dashboard.

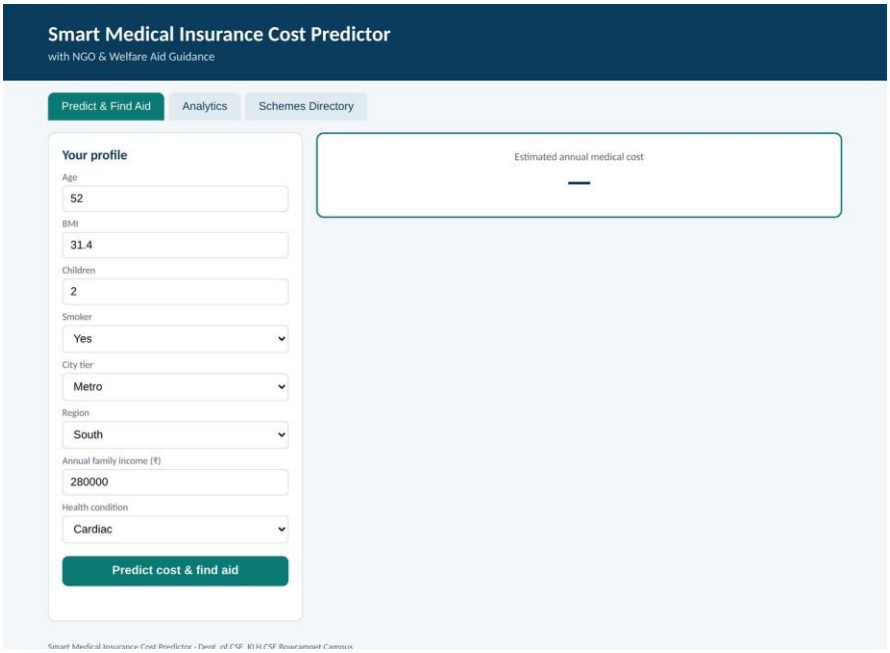



Fig No. A.1 Home screen with the empty form

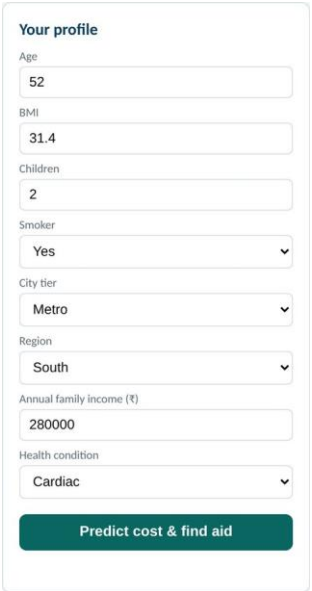



Fig No. A.2 Profile form

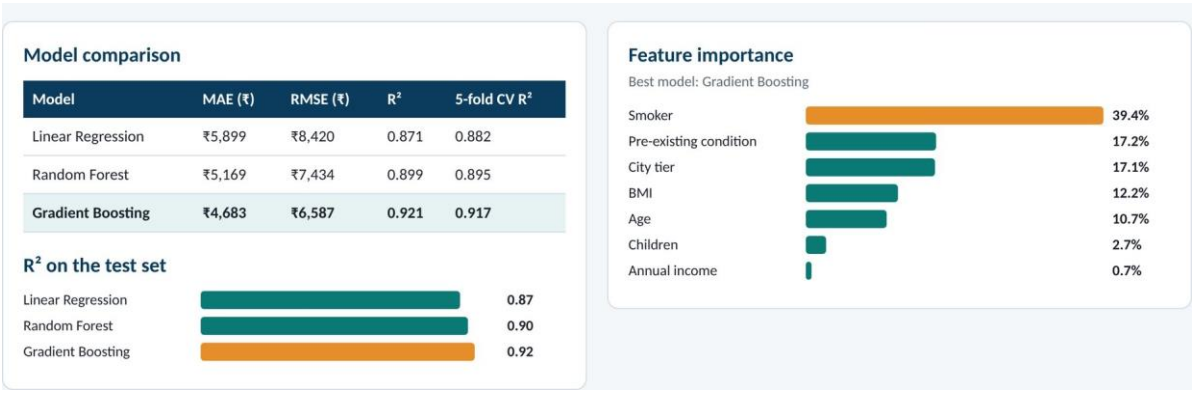



Fig No. A.3 Analytics panel (model comparison, RZ bars and feature importance)

# B. Sample Code Snippets

## B.1 Configuration (src/config.py)

```
SEED = 42
N_RECORDS = 2500

NUM_COLS = ["age", "bmi", "children", "annual_income"]
CAT_COLS = ["smoker", "city_tier", "pre_existing"]
FEATURES = NUM_COLS + CAT_COLS          # 7 inputs used by the models
TARGET = "charges"
REGIONS = ["North", "South", "East", "West", "Central"]
CITY_TIERS = ["Metro", "Tier-2", "Tier-3"]
CONDITIONS = ["none", "cardiac", "cancer", "kidney", "diabetes", "emergency", "surgery", "other"]
```

## B.2 Data generation (src/generate_data.py)

```
def generate(n=N_RECORDS, seed=SEED):
    rng = np.random.default_rng(seed)
    age = np.clip(rng.normal(38, 12, n), 18, 64).round().astype(int)
    sex = rng.choice(["Male", "Female"], n)
    bmi = np.clip(rng.normal(25.6, 4.6, n), 15, 46).round(1)
    children = np.clip(rng.poisson(1.3, n), 0, 4)
    smoker = rng.choice(["Yes", "No"], n, p=[0.14, 0.86])
    region = rng.choice(["North", "South", "East", "West", "Central"], n, p=[0.24, 0.26, 0.18,
0.22, 0.10])
    tier = rng.choice(["Metro", "Tier-2", "Tier-3"], n, p=[0.30, 0.38, 0.32])
    income = (np.exp(rng.normal(np.log(320000), 0.7, n)) / 1000).round() * 1000
    pre = rng.choice(["Yes", "No"], n, p=[0.22, 0.78])

    s = (smoker == "Yes").astype(int)
    p = (pre == "Yes").astype(int)
    tier_factor = np.select([tier == "Metro", tier == "Tier-2"], [1.25, 1.0], 0.85)
    base = (12000 + 650 * age + s * 38000 + np.maximum(bmi - 25, 0) * 2200
            + s * np.maximum(bmi - 30, 0) * 5500 + children * 3500 + p * 22000)
    charges = (base * tier_factor * np.exp(rng.normal(0, 0.09, n))).round(-1)

    df = pd.DataFrame(dict(age=age, sex=sex, bmi=bmi, children=children, smoker=smoker,
region=region,
                           city_tier=tier, annual_income=income.astype(int), pre_existing=pre,
charges=charges))
    # real-world style gaps: 3 percent of BMI values are missing
    miss = rng.choice(n, int(0.03 * n), replace=False)
    df.loc[miss, "bmi"] = np.nan
    return df


if __name__ == "__main__":
    df = generate()
    RAW_CSV.parent.mkdir(exist_ok=True)
    df.to_csv(RAW_CSV, index=False)
    print(f"saved {len(df)} rows to {RAW_CSV}")
```

**B.3 Tests for the matching rules (tests/test_matching.py, extract)**

```
class TestMatchingRules(unittest.TestCase):
    def test_tc01_income_equal_to_limit_is_shown(self):
        p = validate_profile({**BASE, "annual_income": 300000})
        self.assertEqual(len(match(p, 150000, scheme(income_limit=300000))), 1)

    def test_tc02_income_above_limit_is_skipped(self):
        p = validate_profile({**BASE, "annual_income": 300001})
        self.assertEqual(match(p, 150000, scheme(income_limit=300000)), [])

    def test_tc03_any_and_all_match_every_profile(self):
        for region in ["North", "South", "East", "West", "Central"]:
            for cond in ["none", "cardiac", "cancer"]:
                p = validate_profile({**BASE, "region": region, "condition": cond})
                self.assertEqual(len(match(p, 150000, scheme())), 1)
```

## C. Installation and Setup Instructions

- Install Python 3.11 and Git.
- Clone the repository from GitHub and open the project folder in a terminal.
- Run `pip install -r requirements.txt`.
- Run `python -m src.train`. This writes the datasets, the saved model and the metrics file (about half a minute).
- Run `python -m unittest discover -s tests -t . -v`. All 25 tests should pass.
- Run `python app.py` and open http://127.0.0.1:5000 in a browser.

## D. User Manual

- **Predict and Find Aid tab.** Enter age (18 to 100), BMI, number of children, smoker (Yes/No), city tier, region, annual income in ₹ and the condition type (choose "none" if there is no condition). Press the button.
- **Reading the result.** The large figure is the estimated yearly medical cost in ₹. Below it each card gives a scheme, its type, its rules, the documents to carry and the likely aid. The scheme with the largest likely aid is first.
- **If an error appears.** Correct the field named in the message and submit again.
- **Analytics tab.** Shows how the three models compare and which inputs affect cost the most.
- **Schemes Directory tab.** Lists every scheme and its rules so a result can be explained.
- **Remember.** The estimate is a rough guide, the scheme entries are samples, and the organisation makes the final decision on aid.